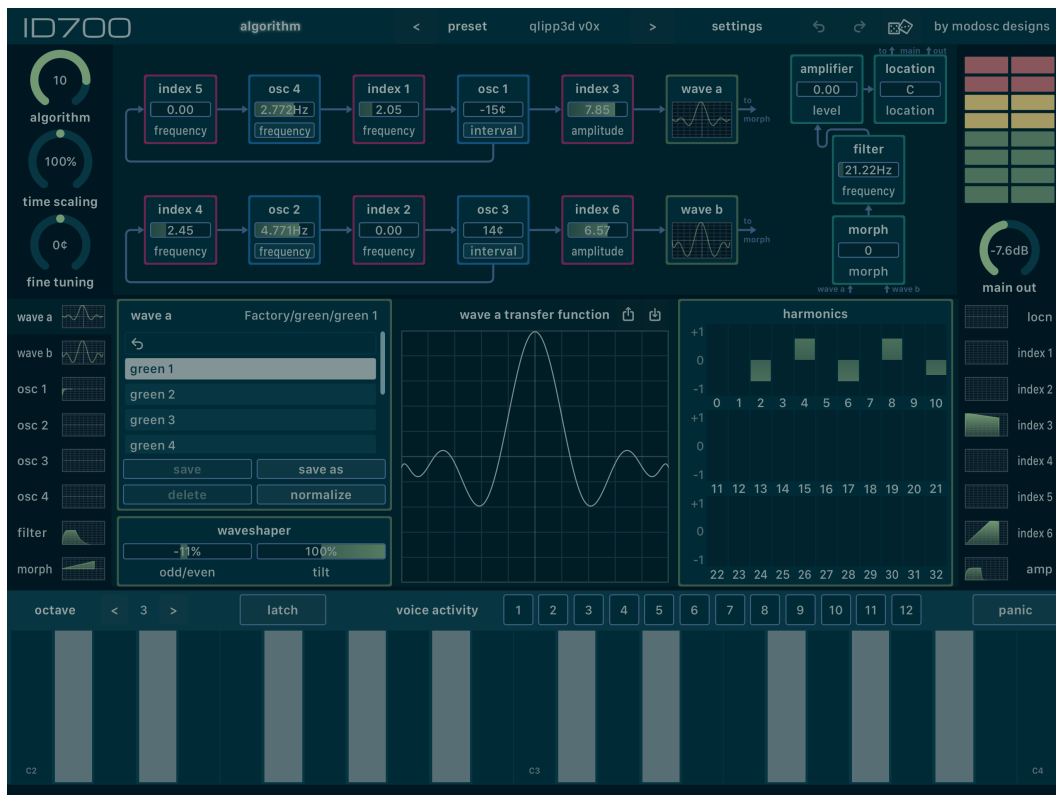


ID700 User Manual

Modosc Designs



Contents

1	Introduction	7
2	System Requirements	8
3	Overview	9
3.1	Interface	9
3.2	Standalone	9
3.3	AUv3 Default	10
3.4	AUv3 “Fullscreen”	10
3.5	Undo/Redo	10
3.6	Preset Randomization	10
4	Algorithm View	11
4.1	Algorithm	11
4.2	Time Scaling	12
4.3	Fine Tuning	12
4.4	Main Out	12
5	Modules	13
5.1	Oscillators	13
5.1.1	Mode	13
5.1.2	Sync	14
5.2	Indexes	14
5.2.1	Amplitude	14
5.2.2	Frequency	15
5.2.3	Timbre	15
5.2.4	MIDI	15
5.3	Wave	15
5.3.1	Transfer Function	15
5.4	Morph	19
5.5	Filter	19
5.6	Amplifier	20
5.7	Location	20
6	Wave/Modulation View	21
6.1	Wave	21
6.1.1	.idwave Files	21
6.1.2	Librarian	21
6.1.3	Transfer Function	22
6.1.4	Copy/Paste	22
6.1.5	Harmonics	23
6.1.6	Waveshaper	23
6.2	Modulation	24
6.2.1	The Problem	25
6.2.2	Proposal 1	25
6.2.3	Proposal 2	26
6.2.4	A Compromise	26

6.2.5	But Why?	27
6.2.6	Module Specific	28
6.2.7	Modulation	28
6.2.8	Curve	28
6.2.9	Random Rates	28
6.2.10	Envelope Drawing	28
6.2.11	Copy/Paste	29
6.2.12	Envelope Randomization	29
6.2.13	Point	29
6.2.14	Point Modulation	29
6.2.15	Conditioned Actions	30
6.2.16	Sources	31
7	Info View	33
7.1	About	33
7.2	Documentation	33
7.3	Debugger	33
8	Controller View	34
8.1	Octave	34
8.2	Latch	34
8.3	Voice Activity	34
8.4	Panic	34
8.5	Keyboard	34
8.5.1	Pitch Bend	34
8.5.2	Timbre	34
8.6	MIDI Out	34
9	Preset View	36
9.1	.idpreset Files	36
9.2	.idpresetpack Files	36
9.3	Browser	36
9.3.1	Save	36
9.3.2	Save As	36
9.3.3	Delete	36
9.3.4	Share	37
9.3.5	Import	37
9.3.6	Export	37
9.4	File	37
9.5	MIDI Sources	37
9.6	Description	37
10	Settings View	38
10.1	Audio	38
10.1.1	Output	38
10.1.2	Sample Rate	38
10.1.3	Audio Buffer Size	38
10.1.4	Test	38
10.2	MIDI	39

10.2.1	MIDI Mode	39
10.2.2	MIDI Channel	39
10.2.3	MPE Mode	39
10.2.4	Pitch Bend Range	39
10.2.5	Slider CC	39
10.2.6	Keyboard Range	39
10.2.7	Active MIDI Inputs	39
10.2.8	MIDI Output	40
10.2.9	Bluetooth Midi	40
10.3	Tunings	40
10.3.1	Import	40
10.3.2	Install	40
10.3.3	Browser	40
10.3.4	File	40
10.3.5	Scale Length	41
10.3.6	Description	41
10.3.7	Pitches	41
10.4	Advanced	41
10.5	Animations	41
10.6	Envelope Quality	41
10.7	Oversampling	41
10.8	Voice Mode	42
10.8.1	Polyphonic	42
10.8.2	Monophonic	42
10.9	Preset Auto Panic	43
10.10	Rotary Slider Mode	43
10.11	Slider Fine Tuning Mode	43
10.12	iCloud	44
11	MIDI Implementation	45
11.1	Program Change	45
11.1.1	Factory Presets	45
11.1.2	User Presets	45
11.2	Thru	45
11.3	Continuous Parameters	45
11.4	On / Off Parameters	45
11.5	Control Numbers	46
12	.idwave File Format	48
12.1	<Waveshape>	49
12.2	<Harmonics>	49
12.3	<Harmonic>	49
12.4	DTD	49
13	Envelope Format	50
13.1	<Envelope>	50
13.1.1	id	50
13.1.2	curve	50

13.1.3	generalSource	51
13.1.4	generalSourceMult	51
13.1.5	smoothedRandomRate	51
13.1.6	steppedRandomRate	51
13.1.7	active	51
13.2	<Segment>	51
13.2.1	time	52
13.2.2	level	52
13.2.3	source	52
13.2.4	sourceMult	52
13.2.5	conditionedActionType	52
13.2.6	conditionedActionIterator	52
13.2.7	conditionedActionTarget	53
13.2.8	DTD	53
14	Factory Presets	54
14.1	Bank 00 Programs	55
14.2	Bank 01 Programs	56
15	About	57
15.1	Credits	57
15.1.1	Testers	57
15.1.2	Presets	57
15.1.3	Editor	57
15.1.4	Tunings	57
15.1.5	Waveshapes	57
15.1.6	Icons	57
15.1.7	Third-Party Software	57
15.1.8	Special Thanks	58
15.1.9	Extra Special Thanks	58
15.1.10	Shoutouts	58
16	Changelog	59
[1.30.0]	- 2025-10-18	59
[1.29.0]	- 2024-03-22	59
[1.28.0]	- 2024-03-09	59
[1.27.0]	- 2023-12-17	59
[1.26.0]	- 2023-08-28	59
[1.25.0]	- 2023-07-30	59
[1.24.0]	- 2023-07-09	59
[1.23.0]	- 2023-06-10	59
[1.22.0]	- 2023-06-04	59
[1.21.0]	- 2023-02-22	59
[1.20.0]	- 2023-02-16	60
[1.19.0]	- 2023-01-22	60
[1.18.0]	- 2022-09-11	60
[1.17.0]	- 2022-03-26	60
[1.16.0]	- 2022-02-23	60

[1.15.0] - 2022-02-12	60
[1.14.0] - 2022-01-20	60
[1.13.1] - 2022-01-08	60
[1.13.0] - 2021-12-22	60
[1.12.2] - 2021-12-19	61
[1.12.1] - 2021-12-17	61
[1.12.0] - 2021-11-21	61
[1.11.0] - 2021-10-07	61
[1.10.0] - 2021-07-05	61
[1.9.0] - 2021-06-14	61
[1.8.0] - 2021-06-04	61
[1.7.0] - 2021-05-24	61
[1.6.0] - 2021-05-18	61
[1.5.0] - 2021-05-10	62
[1.4.0] - 2021-04-30	62
[1.3.0] - 2021-04-23	62
[1.2.0] - 2021-04-19	62
[1.1.0] - 2021-04-14	62
[1.0.0] - 2021-04-08	62

17 End-User License Agreement (EULA) of ID700	63
License Grant	63
Intellectual Property and Ownership	64
Termination	64
Governing Law	64

1 Introduction

In 1989 Don Buchla released the Buchla 700, which was (and still is) the most sophisticated instrument he ever designed. It featured twelve multitimbral digital voices with complex modulation possibilities. It also included a score editor, a sequencer, custom tunings, wave editing, and more in a physical package designed to fit underneath an airline seat (for the touring musician who doesn't trust checked baggage). The 700 shipped with extensive I/O including multiple MIDI ports (it was Buchla's first MIDI enabled instrument), serial ports, and banana jacks for control voltage from Don's other instruments. The 700 was controlled by the MIDAS-VII software, but in theory users could also design their own OS (there was precedent for this with the Buchla 400 which had two different OSs available).

Overall, the specs of the Buchla 700 were groundbreaking (both in 1989 as well as now), so why wasn't it a huge success? Unfortunately the 700's design had a single source of failure in the external display. Most of the 700's editing functionality was only available through the external monitor interface. This was by design; the idea was for a composer to build up their compositions at home with the connected monitor, then take the relatively smaller and lighter 700 with them for performances. The touch surface offered plenty of controls appropriate for a live performance, but without a display the rest of the parameters were effectively hidden. This could have been merely an annoyance—both the Touché and the Buchla 400 used similar setups for the same reason—but the 700's design combined with the Intel video chip Don chose resulted in a high rate of failure. Once the video card failed, the 700 essentially became an overpowered preset synth, and that's assuming the owner had actually programmed any presets, as the 700 shipped with a single preset named “De Fault”. Overall, there were approximately fourteen Buchla 700s built, and as far as I know there are only two currently working units left (and “working” is a stretch, as both units need constant maintenance).

This brings us to the near-present. Don passed in 2016, and in 2017 there was a set of memorial concerts in San Francisco. As part of these performances a collection of Don's instruments was displayed, and it was there that I had my only interaction with a Buchla 700. The room was cramped and noisy and it was hard to make out anything specific but I became enamored with the 700. I decided that I wanted to build a version of it for fun, mostly to improve on the user interface. I initially set out to build a faithful 1:1 copy, but I realized that any copy I made would be just as difficult to use as the original. I eventually decided to focus only on the synthesis portion of the instrument; the sequencing/scoring/preset management functionality could all be better handled by any modern DAW, but the sound generation was still unique. ID700's basic design principle is pragmatism rather than dogmatism, and the result is a usable modern interpretation of the Buchla 700 rather than a clone or emulation.

2 System Requirements

The following table summarizes the operating system requirements and provided formats for ID700

Operating System	Minimum Version	Formats	Notes
iOS	iOS 11 or iPadOS 13	AUv3, standalone app with IAA	iOS versions require an iPad
MacOS	10.13	AUv3, standalone app, VST3 wrapper	MIDI Output is currently disabled on MacOS pending a fix from Apple.

3 Overview

ID700 is a software synthesizer based on the voice architecture of the Buchla 700. Like many of Don's other designs, sound generation is achieved primarily via FM synthesis and waveshaping. Each voice is made up of sixteen individual modules:

- four **Oscillators**
- six **Indexes**
- two **Waves**
- one **Morph**
- one **Filter**
- one **Amplifier**
- one **Location**

The **Oscillators**, **Indexes**, and **Waves** are configured in one of twelve preset **Algorithms**, all of which feed into the fixed signal paths of the **Morph**, **Filter**, **Amplifier**, and **Location** modules.

Each of the **Oscillator**, **Index**, **Morph**, **Filter**, **Amplifier**, and **Location** modules have their own dedicated complex **Envelope** for modulation.

The **Wave** modules provide wavetable synthesis using Chebyshev polynomials and include additional waveshaping functionality.

Arbitrary **Tunings** are supported via the Scala Tuning File format.

MIDI and MPE are supported (note: not all hosts support MPE).

3.1 Interface

ID700's main interface includes six main views:

- **Algorithm**
- **Controller**
- **Info**
- **Preset**
- **Settings**
- **Wave/Modulation**

There are three different layouts of these views which depend on how ID700 is running.

3.2 Standalone

When running as a standalone application there are three vertically stacked panes. The top pane is used for one of the following:

- **Algorithm**
- **Info**
- **Preset**
- **Settings**

To select a different view, click the associated label in the header.

The middle pane is used for **Wave/Modulation**, and the bottom pane is used for the **Controller**.

3.3 AUv3 Default

AUv3 plugins on iOS default to a smaller screen size. In this mode there is only a single pane. All views are accessed here by selecting them from the header bar.

3.4 AUv3 “Fullscreen”

Some AUv3 hosts offer a larger “fullscreen” view of a plugin. This mode is identical to the Standalone mode but only has two vertically stacked panes instead of three. The **Controller** is moved into the top pane and is accessible from the header bar.

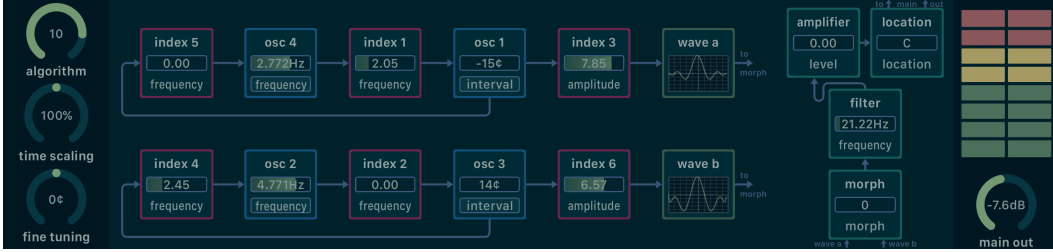
3.5 Undo/Redo

ID700 supports multiple undo/redo layers. The current state of the undo/redo stack is displayed via the two curved arrows in the header bar. When an undoable change exists, the arrow pointing to the left will be enabled, and when a redoable change exists the arrow pointing to the right will be enabled. The undo/redo stack is also cleared every time a new **Preset** is loaded. If you are using a keyboard then you can also access this functionality via the standard system shortcuts (**Command-Z** and **Shift-Command-Z**)

3.6 Preset Randomization

Clicking on the pair of dice in the header bar will randomly alter the currently loaded **Preset**. If you are using a keyboard then you can also access this functionality via **Command-R**. **Preset Randomization** responds to MIDI, see the MIDI section for more details.

4 Algorithm View



The **Algorithm** view provides a view of ID700's signal flow as well as several performance parameters.

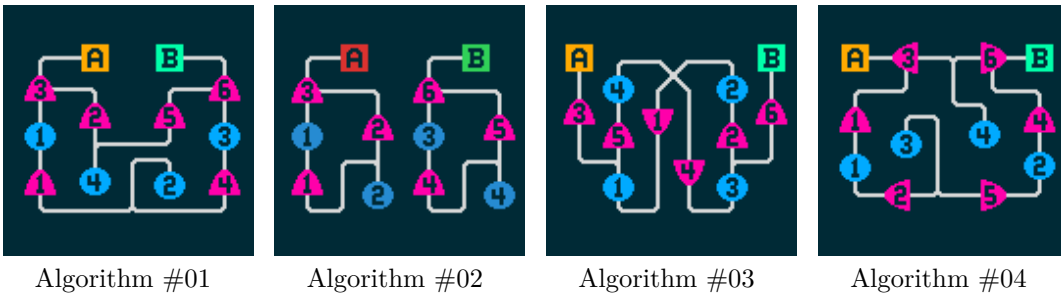
4.1 Algorithm

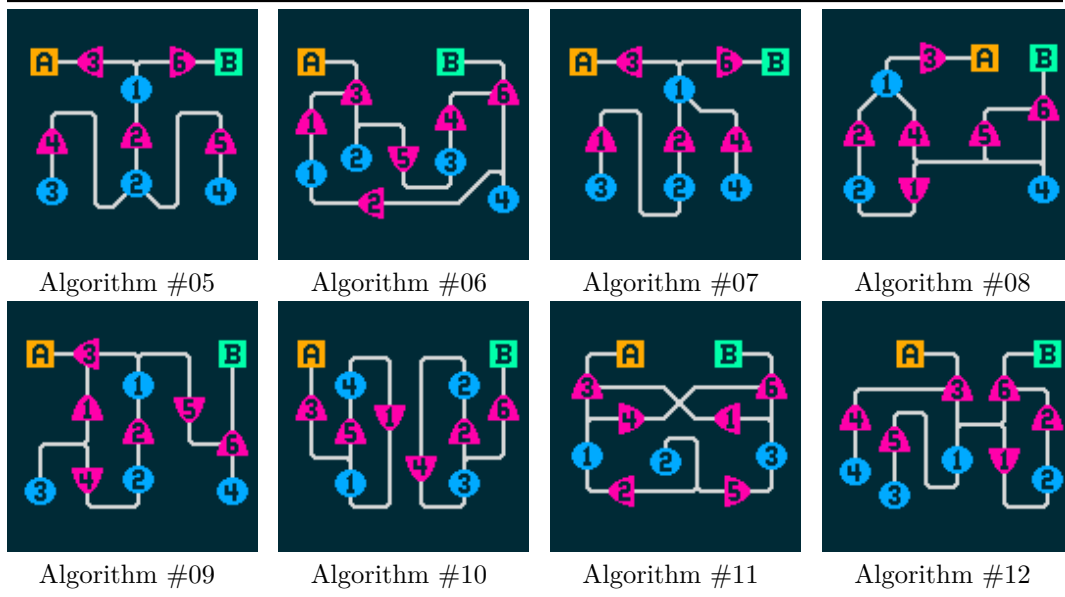
The **algorithm** parameter selects the **Algorithm** to use for sound generation. ID700's twelve **Algorithms** are identical to the Buchla 700's. Each **Algorithm** is a pre-patched layout of four **Oscillators**, six **Indexes**, and two **Waves**. Note that some **Algorithms** don't use all of the **Oscillators** or **Indexes**. The current **Algorithm**'s signal flow is displayed in the center of the screen. Signal flows follow the drawn paths in the direction indicated by arrows. There are a few special cases:

- **Indexes** may also have connections perpendicular to the signal flow - these are **Timbre** modulation inputs and are discussed below.
- The outputs from both **Wave** modules are always connected to the **Morph** module's inputs. These are not explicitly drawn for layout reasons, but are labelled as such.

Most modules have a primary parameter that can be controlled by the slider in the module's view (the exceptions are the **Wave** modules which instead display their current transfer function). Clicking anywhere on the module other than the slider will select that module for editing in the **Wave/Modulation** section (if you are in the default AUv3 view this will also navigate away from the **Algorithm** view). The functionality of each module is discussed in-depth in the **Modules** section.

The original twelve **Algorithms** from the Buchla 700 are shown below. ID700's **Algorithms** are functionally identical but drawn differently for usability.





4.2 Time Scaling

The `time scaling` parameter allows all time-dependent functions to be scaled from 25% to 400%. At 25% an event that happens every 100ms will occur every 25ms (i.e., four times as fast), while at 400% the same event will occur every 400ms (i.e., four times as slow). This parameter is stored as part of the current **Preset** and affects the following values:

- Envelope Points
- Stepped Random Rate
- Smooth Random Rate

`time scaling` responds to MIDI, see the MIDI section for more details.

4.3 Fine Tuning

ID700 is tuned by default to A440. The `fine tuning` parameter can be adjusted +/- 100 cents and is stored as part of the current **Preset**.

`fine tuning` responds to MIDI, see the MIDI section for more details.

4.4 Main Out

The `main out` parameter adjusts the total output level of all summed **Voices** from -100dB to 6dB, with 0dB resulting in no additional gain.

The area above the `main out` knob is a sound level meter which shows the output levels for the left and right channels.

`main out` responds to MIDI, see the MIDI section below for more details.

5 Modules

5.1 Oscillators



In the **Algorithm** view **Oscillators** are represented as blue squares labelled “osc 1” through “osc 4.” Inputs to the bottom of an **Oscillator** will modulate the **Oscillator**’s frequency, and multiple inputs to an **Oscillator** are summed. ID700’s **Oscillators** produce only sine waves.

Each Oscillator responds to MIDI, see the MIDI section for details.

5.1.1 Mode

The frequency slider on each **Oscillator** behaves differently depending on which mode is selected.

5.1.1.1 Interval

In **Interval** mode the **Oscillator**’s frequency is determined by the incoming MIDI note’s frequency offset by the slider value. The slider value is quantized in cents and is adjustable by +/- 2 octaves (+/- 2400 cents).

5.1.1.2 Ratio

In **Ratio** mode the **Oscillator**’s frequency is determined by the incoming MIDI note’s frequency multiplied by the slider value. The slider value is quantized to rational numbers and has a linear range of 1/16-8/1.

5.1.1.3 Frequency

In **Frequency** mode the **Oscillator** will ignore the pitch of incoming MIDI notes. The slider sets the frequency in a subaudio range of 0.1mHZ-16.636HZ. This mode only produces audible sound at the high end and is primarily useful for modulation.

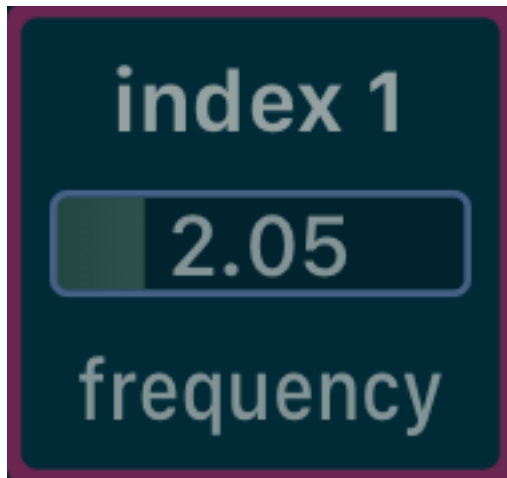
5.1.1.4 Pitch

In **Pitch** mode the **Oscillator** will ignore the pitch of incoming MIDI notes. The slider sets the frequency in a range of C0.00-C9.00. The characteristic (portion to the left of the decimal point) represents the MIDI note used as the base frequency, and the mantissa (portion to the right of the decimal point) is a positive offset to the note's pitch measured in cents.

5.1.2 Sync

Oscillators 1-3 can each be synchronized to **Oscillator** 4. When this mode is enabled the synchronized **Oscillator(s)** will reset every time **Oscillator** 4 starts a new cycle. The **Sync** button is accessed in the Modulation view for the **Oscillator**.

5.2 Indexes



Indexes are linear amplifiers which allow dynamic control over the signal path of an **Algorithm**. In the **Algorithm** view **Indexes** are represented as pink boxes labelled “index 1” through “index 6.” **Index** inputs normally connect to the bottom of an **Index**, and outputs exit from the top. **Timbre** indexes are the exception to this rule - they connect horizontally to the **Amplitude** index that they are modulating.

Each **Index** has a range of 0-10. A value of 0 prevents a signal from passing, and a value of 10 passes the incoming signal without modification (unity gain).

Each **Index** has one of three different types, which are displayed under the slider area of the **Index**. The possible types are:

5.2.1 Amplitude

Amplitude Indexes sit between an **Oscillator**'s output and a **Wave**'s input. This type of **Index** controls the amplitude of the input signal entering the **Wave** module. In general, increasing this type of **Index**'s value will increase the harmonic complexity of the **Wave**'s output. When this value is zero, the **Wave**'s output will be nearly identical to the input value. As the **Index**'s value rises the **Wave**'s harmonic content will approach the levels set in the **Wave**.

5.2.2 Frequency

Frequency Indexes sit between an Oscillator's output (modulator) and another Oscillator's input (carrier). This type of Index controls the modulation index of the modulator applied to the carrier. When this value is zero, no modulation occurs, and as the level increases the amount of modulation increases. This Index's output is bipolar.

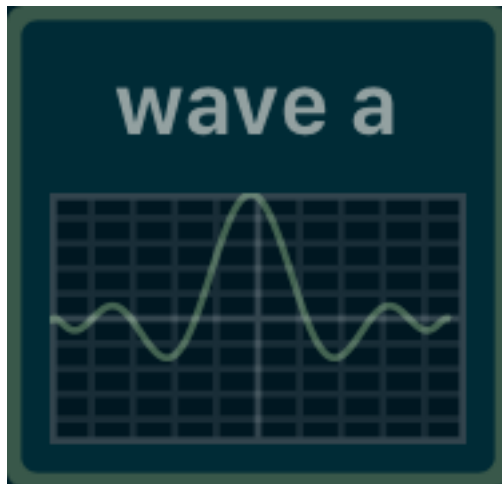
5.2.3 Timbre

Timbre Indexes sit between an Oscillator's output (modulator) and an Amplitude Index's input (carrier). This type of Index modulates the level of an Amplitude Index. Specifically, the Amplitude Index's level is multiplied by a phase-shifted version of the Timbre Index's output. This is not true amplitude modulation (AM) because the carrier's signal is never increased. This results in clipping once the sum of the two Indexes is above ten. Instead, the incoming modulator signal inversely applies gain to the carrier signal, reducing it from full amplitude to zero and then inverting the signal. If the connected Wave module is using a Waveshape with only a single Harmonic, the output is indistinguishable from AM, but if the Waveshape is using more than one Harmonic then this results in very complex harmonic output. This Index's output is bipolar.

5.2.4 MIDI

Each Index responds to MIDI, see the MIDI section for more details.

5.3 Wave



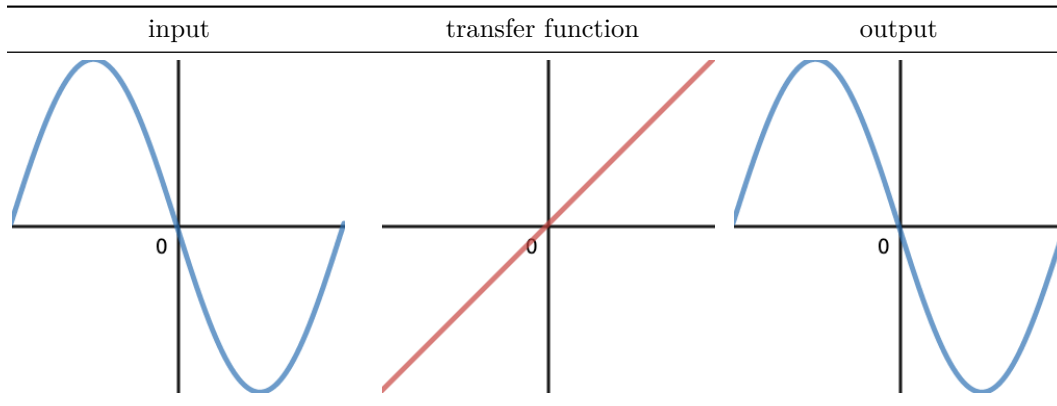
A Wave modifies an incoming waveform with a transfer function and outputs the result to the Morph.

5.3.1 Transfer Function

The graphs displayed in the Wave sections are graphs of the transfer function. A transfer function modifies the incoming signal to produce a new output. This is a difficult concept to grasp but some examples will help a bit. For all of the examples our input value is a simple

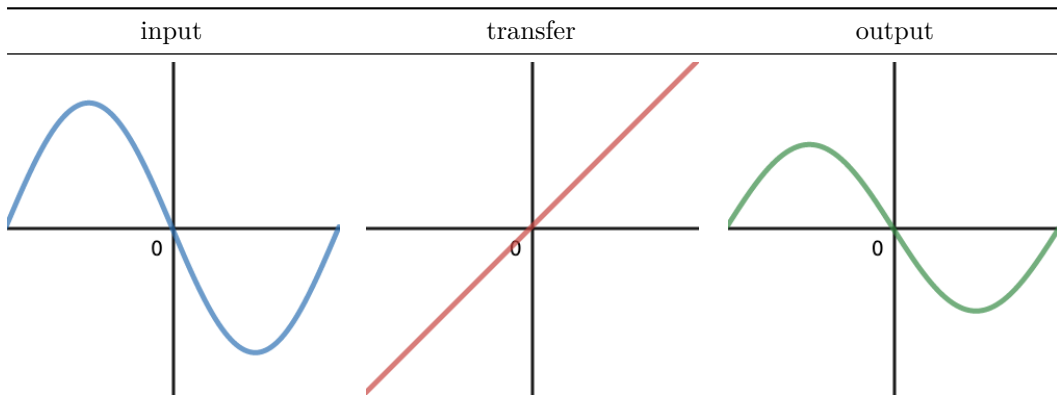
sinusoid, as this is identical to ID700's synthesis when no FM is applied to the `Oscillators`. If you're mathphobic (or in a hurry), you can skip to the end of the examples for a summary.

5.3.1.1 Example 1



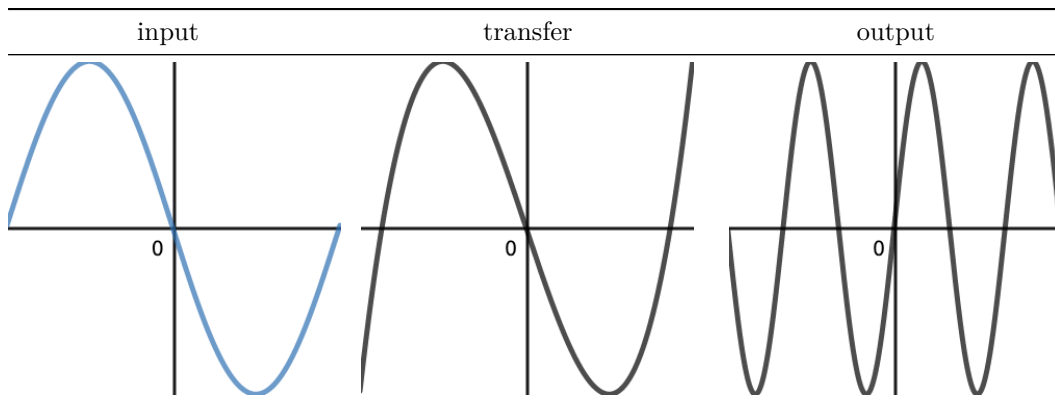
This is the simplest transfer function possible. It passes the input signal through unmodified, which in this case results in another sinusoid wave.

5.3.1.2 Example 2



If the input signal's amplitude is reduced the output signal is also affected. In ID700 this technique is accomplished with **Amplitude Indexes** (which directly control the level of a **Wave's** input signal). In this example the output signal's amplitude is also reduced, but as we'll see that's not always the case.

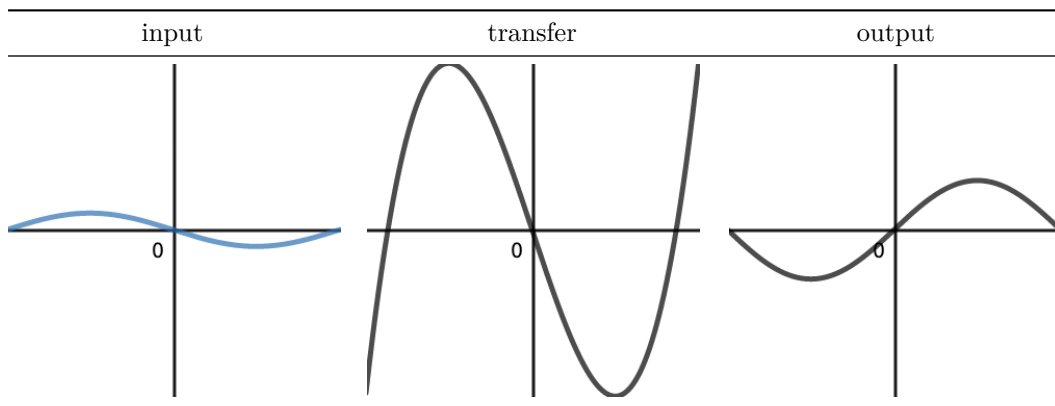
5.3.1.3 Example 3



A more complex transfer function gives a more complex output. In this example our transfer function is the third Chebyshev polynomial. The resulting output is a sinusoid with three times the frequency of the input signal, also known as the third harmonic. A detailed discussion of Chebyshev polynomials is beyond the scope this documentation, but **the important takeaway here is the Nth Chebyshev polynomial will convert a full-amplitude sinusoid into that sinusoid's Nth harmonic.** This is the basis of ID700's wavetable synthesis.

This transfer function can be created in ID700 by editing a Wave to have Harmonic 3 at -1 or +1 and all other Harmonics at 0.

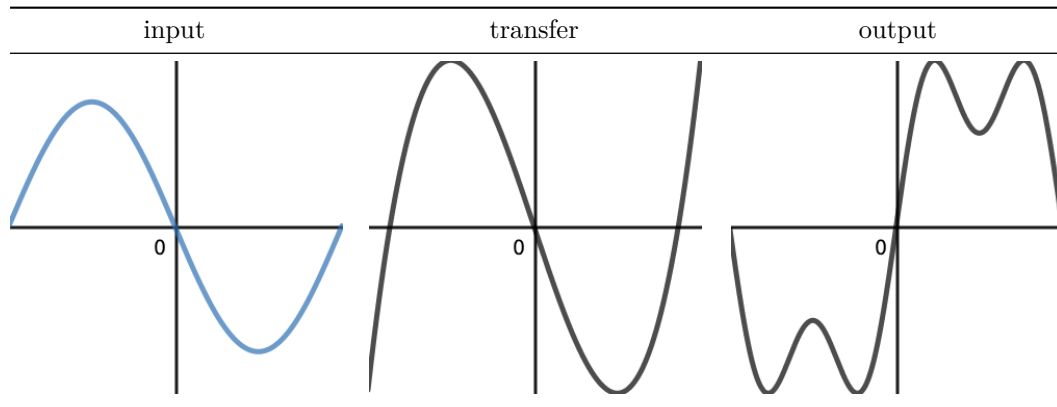
5.3.1.4 Example 4



In Example 2 we hinted that reducing the amplitude of a transfer function's input value doesn't always modify the output in the same way. Here we're still using the third Chebyshev polynomial as our transfer function, but we've reduced our input sinusoid to 10% of the original value. The output value is also a sinusoid, but with an amplitude that's greater than the input value.

The takeaway here is **the Nth Chebyshev polynomial will convert a low-amplitude sinusoid into another low-amplitude sinusoid with the same frequency.**

5.3.1.5 Example 5



In Example 4 we showed that greatly reducing the amplitude of our input value results in a similarly reduced output. This behavior is not consistent across the amplitude range of our input value, and here we see a more interesting example. Our transfer function is still the third Chebyshev polynomial but now we've reduced the input sinusoid's amplitude to 75% of the original value. Unlike our previous example, the output is now something quite different from both the input and the transfer function. It has a greater amplitude and contains more harmonic content (i.e., the lines are more squiggly). It turns out that the harmonic content of our output is made up of various sums of the first and second harmonics of our input sinusoid.

The takeaway here is **the Nth Chebyshev polynomial will convert a medium-amplitude sinusoid into some combination of the input signal's harmonics up to (but not including) N.**

5.3.1.6 Summary

1. A small input signal from an **Amplitude Index** will result in an output with very few harmonics.
2. A large input signal from an **Amplitude Index** will result in an output with harmonic content matching that of the **Wave**.
3. Anything in between will result in interesting harmonics that are not the input signal but also not the target output.

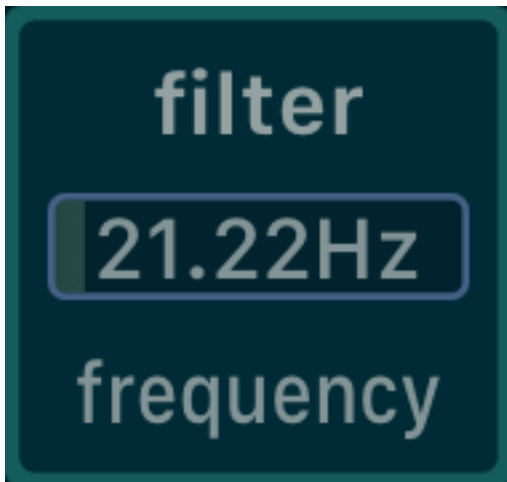
5.4 Morph



The **Morph** is an equal-power fader between the outputs of **Wave A** and **Wave B**. The far left position is 100% **Wave A**, the far right is 100% **Wave B**, and the center position is an equal mix of both **Waves**.

The **Morph** responds to MIDI, see the MIDI section for more details.

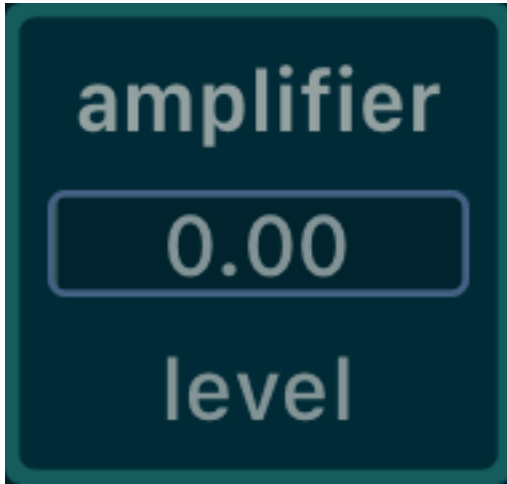
5.5 Filter



The **Filter** is a -24db/oct resonant low-pass filter. The **cutoff frequency** range is exponential from 20Hz-20kHz and the **resonance** range is linear from 0-10. The **resonance** control is accessed in the **Modulation** section for the **Filter**.

The **Filter** responds to MIDI, see the MIDI section for more details.

5.6 Amplifier



The **Amplifier** is a linear amplifier with a range of 0-10. A value of 0 prevents a signal from passing, and a value of 10 passes the incoming signal without modification.

The **Amplifier** responds to MIDI, see the MIDI section for more details.

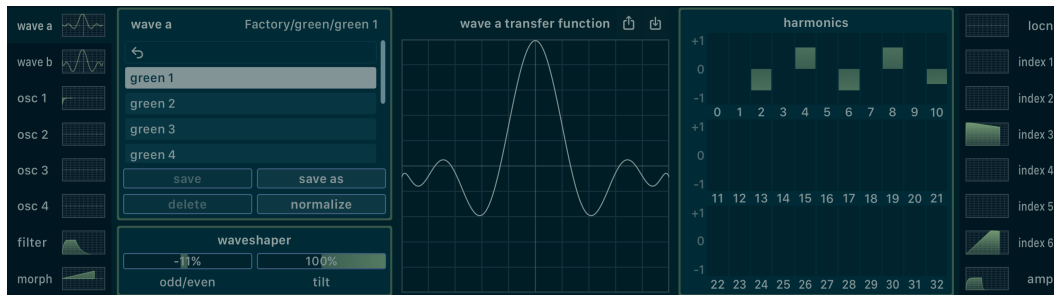
5.7 Location



The **Location** is an equal-power fader between the left and right outputs. The slider's position represents the output's location in the stereo field.

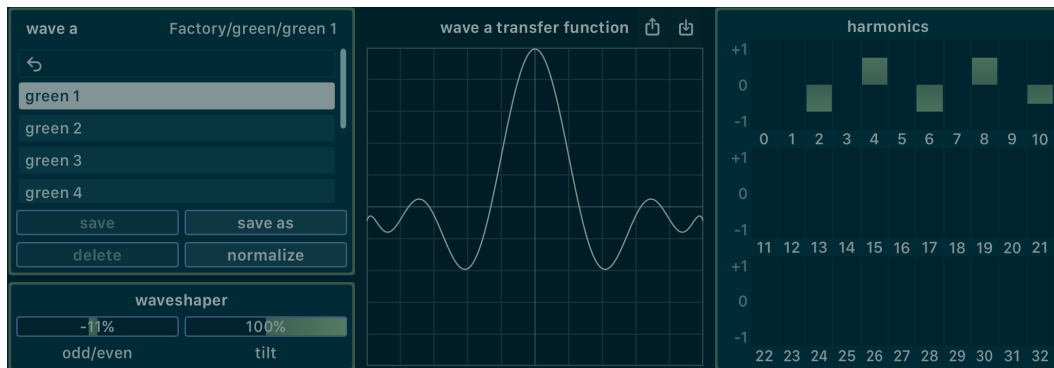
The **Location** responds to MIDI, see the MIDI section for more details.

6 Wave/Modulation View



This section gives a thumbnail overview of both **Waves** and all fourteen **Envelopes**. Clicking one of these thumbnail sections will open up the associated editor view in the center.

6.1 Wave



Clicking the **Wave** display will open the editor interface. Here you can modify a **Waveshape** in realtime and perform librarian functions. Note that a **Preset** keeps its own copies of a **Waveshape**, so modifications won't affect other **Presets** unless you save and explicitly reload the **Waveshape** in your other **Preset**.

6.1.1 .idwave Files

ID700 **Waveshapes** have the **.idwave** extension (see also the **.idwave File Format**). Double-clicking on an **.idwave** file will import it into the **User/** directory. All **Waveshapes** live in a central directory that is shared between the various formats of the plugin.

6.1.2 Librarian

The librarian section is on the left side. This allows you to perform basic file management operations for **Waveshapes**.

6.1.2.1 Selected Waveshape

The top row displays which **Wave** (A or B) you're editing and which file your **Waveshape** was originally loaded from. If the **Waveshape** file doesn't exist on your local file system, an icon

of a folder containing a question mark will be shown to the left side. See the **Save** section below for how to save the file locally.

6.1.2.2 Browser

The browser allows you to select different **Waveshapes**. Clicking a **Waveshape** will load it up in the currently selected **Wave**.

6.1.2.3 Save

Clicking the **save** button will save the current **Waveshape** to disk. Note that this does not affect any other saved **Presets** which reference the **Waveshape** because each **Preset** stores a local copy of its selected **Waveshapes**.

6.1.2.4 Save As

Clicking the **save as** button will bring up a dialog to specify a new name for the currently selected **Waveshape**. If you wish to put the file in a subdirectory you must specify it as part of the name, e.g., “My New Folder/My New Waveshape” (nonexistent directories will automatically be created). Once you click **ok** in the dialog, the file will be saved and your **Preset** will now reference the newly saved **Waveshape** file. Note that your current **Preset** is *not* saved during this action.

6.1.2.5 Delete

Clicking the **delete** button will delete the current **Waveshape** file from disk. Once you click **ok** in the dialog, the file is removed from the file system and your current **Preset** will now reference the next existing **Waveshape** it can find in the current directory. If your file was in a directory that is now empty the directory will also be removed. Note that your current **Preset** is *not* saved during this action.

6.1.2.6 Normalize

It’s quite easy to add too many harmonics to a **Waveshape**. When this happens, the output may clip and produce distortion. You can see this clearly in the transfer function graph as horizontal lines at the bottom or top (-1 or 1). Clipping can also occur when DC offset is introduced, either directly from **Harmonic 0** or indirectly due to the way Chebyshev polynomials work. The normalization function attempts to rescale the given set of harmonics in a **Waveshape** to avoid clipping while still maintaining the same ratio of harmonic content.

6.1.3 Transfer Function

The display in the center is a graph of the transfer function defined in the current **Wave** (see the discussion in the **Wave** section for more details). Portions of the transfer function graph which continue beyond the vertical bounds indicate clipping and should be avoided.

6.1.4 Copy/Paste

To the right of the transfer function’s title are the **copy** and **paste** buttons. Both **copy** and **paste** utilize the system’s clipboard which makes it possible to create and edit **Waveshapes**

outside of ID700. These functions use the .idwave File Format for the **Waveshape** data. To **copy** the current **Waveshape** to the clipboard click the up arrow button, and to **paste** from the clipboard into the current **Waveshape** click the down arrow button.

6.1.5 Harmonics

The right side of the **Wave** editor screen is used to control the harmonic content of the currently selected **Waveshape**. Each slider represents the coefficient for that **Harmonic**. The range is linear from -1 to 1, and negative values invert the phase of the **Harmonic**.

Harmonic 0 is special and represents DC (vertical) offset. Modifying **Harmonic 0** can sometimes help to keep a busy **Waveshape** from clipping. Also note that **Harmonic 0** is not affected by the **Waveshaping** controls. The remaining **Harmonics (1-32)** represent partials (i.e., multiples of the incoming sine wave).

Signification (alternating the sign of harmonics) can help reduce clipping while maintaining identical harmonic content. You may have luck with alternating every harmonic (+,-,+,-) or every other harmonic (+,+,-,-).

6.1.6 Waveshaper

The **Waveshaper** section allows for additional processing of the selected **Waveshape** file.

6.1.6.1 Odd/Even

The **odd/even** parameter alternately applies up to +/-3db gain to the **Harmonics** in a **Waveshape**. A neutral(0%) setting means the overall harmonic content is unchanged. At -100% odd **Harmonics** are boosted and even **Harmonics** are reduced, and at 100% even **Harmonics** are boosted and odd **Harmonics** are reduced. Changes to this parameter are stored as part of a **Preset** but are not stored in a **Waveshape** file.

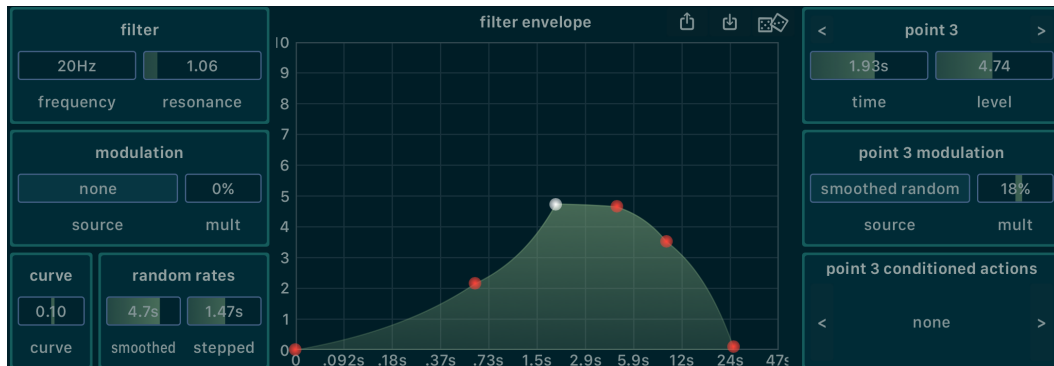
odd/even responds to MIDI, see the MIDI section for more details.

6.1.6.2 Tilt

The **tilt** parameter controls the harmonic tilt of a **Waveshape** by adding up to 3db gain to one set of **Harmonics** while applying an opposite amount of gain to the other. The slope of this tilt is centered between **Harmonics 16** and **17**. At -100% lower **Harmonics** are boosted and higher **Harmonics** are reduced; at 100% the opposite occurs. The gain is most extreme at the far ends, so **Harmonics 1** and **32** can receive up to +/-3db gain, **Harmonics 16** and **17** won't change at all, and the other **Harmonics** receive gain exponentially increasing based on their distance from the center. Changes to this parameter are stored as part of a **Preset** but are not stored in a **Waveshape** file.

tilt responds to MIDI, see the MIDI section for more details.

6.2 Modulation



Each ID700 voice has fourteen complex **Envelopes**, one for every module (excluding both **Waves**).

Each **Envelope** is triggered simultaneously by an incoming MIDI note. A **Voice** is considered “done” when the **Amplitude Envelope** hits a terminal state. Until then, all **Envelopes** will continue to run.

Each **Envelope** is made up of an arbitrary number of **Points**. The **Envelope**’s path generally flows from one **Point**’s value to the next **Point**’s value but this logic can be changed with **Conditioned Actions**. The path an **Envelope** follows between two **Points** is determined by the **Envelope**’s curve value.

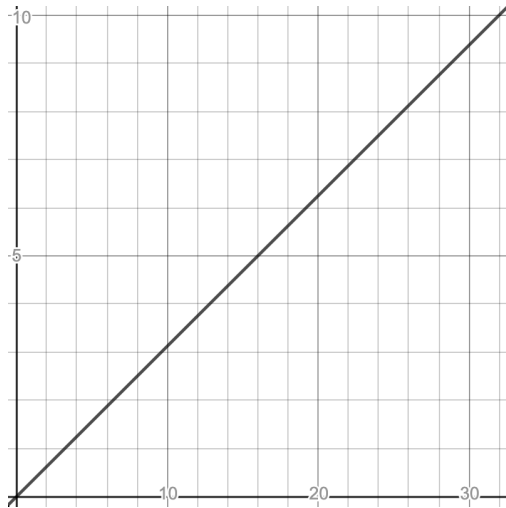
Each **Point** has a **time** (x-axis) value and a **level** (y-axis) value.

The range of an **Envelope** is 0s to 32s and is divided into 10 steps. The range is logarithmic (mostly) which means that the time between adjacent steps is twice as long as the time between the previous steps. The range is also affected by the **time scaling** parameter - when this is used the labels on the x-axis will reflect the newly scaled values.

The domain of an **Envelope** is either 0 to 10 or -5 to 5 (depending on which module the **Envelope** is associated with) and is shown on a graph with 10 vertical steps.

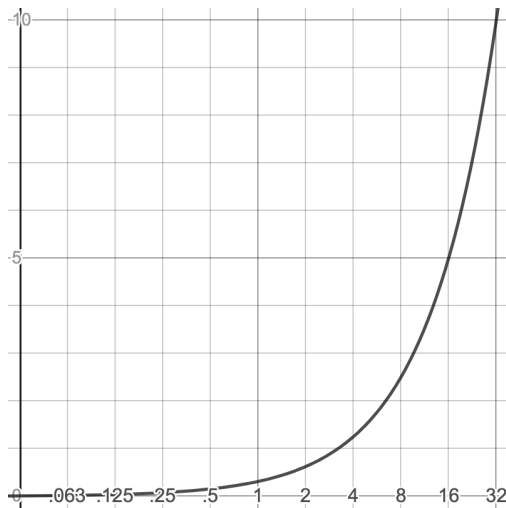
Envelopes are drawn so that the **level** and **time** values match the x and y axis labels. However the *path* of an **Envelope** between two **Points** is not drawn according to the x-axis, but rather according to how the path is traversed. This is because the x-axis is logarithmic instead of linear. The following examples will help explain this difference.

6.2.1 The Problem



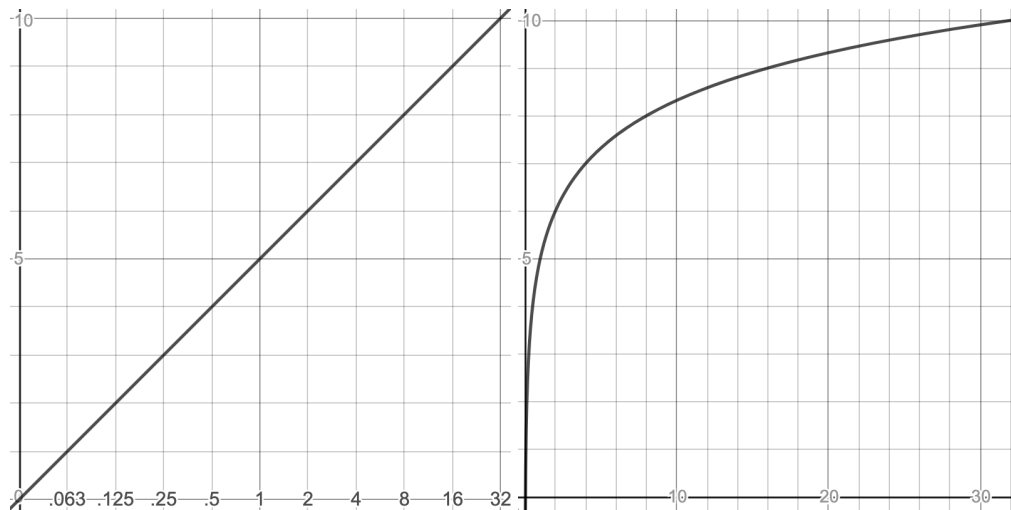
We want to express a value that takes 32 seconds to interpolate linearly between 0 and 10. Linear interpolation means we are increasing our value at a constant rate. The difference between the values at 0s and 1s should be the same as the difference between 1s and 2s. The graph above shows this on a normal (linear scale) graph; the x-axis represents seconds from 0 to 32 and the y-axis represents our output value (0 to 10). ID700 doesn't use a linear scale though, so how can we best express this?

6.2.2 Proposal 1



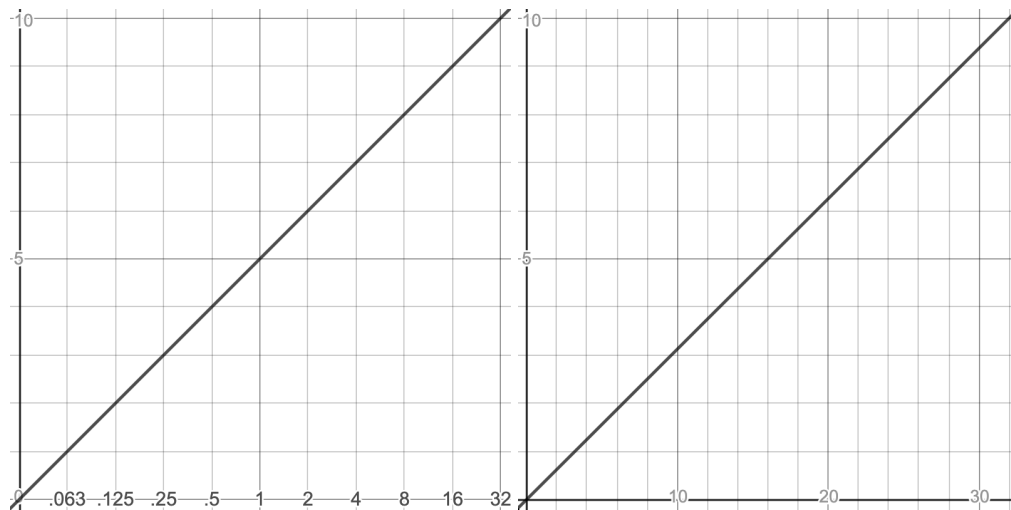
If we take our function and move it to a logarithmic x-axis (like the one ID700 uses) our straight line becomes an exponential curve. We can see that the points on both graphs are more or less the same—e.g., at 16s we'd expect our result to be 5 (halfway between 0 and 10) and this is the case in both graphs. Even so, this could be confusing if we were expecting our linear function to actually look linear.

6.2.3 Proposal 2



Let's try this a different way. We still want to express a linear rise from 0 to 10 in 32 seconds, so what happens if we draw it that way on the logarithmic graph? The first graph expresses the idea of what we want (constant rate interpolation from 0 to 10 in 32 seconds), and the endpoints match up to our expected values, but what about the other points? If we check at 16 seconds, we'd expect our result to be 5, but it's actually 9. If we convert this function to a linear x-axis we discover that our function is now logarithmic. This behavior is not what we wanted, and is also confusing but for different reasons.

6.2.4 A Compromise



It appears as though there's no reasonable way forward here, so we compromise. Consider our logarithmic x-axis graph on the left again. The intent is a linear rise from 0 to 10 over the span of 32 seconds. ID700's **Envelope** drawing/display is only accurate at the end **Points** (in this case, at $[0,0]$ and $[32,10]$). However, the path between the **Points** describes the

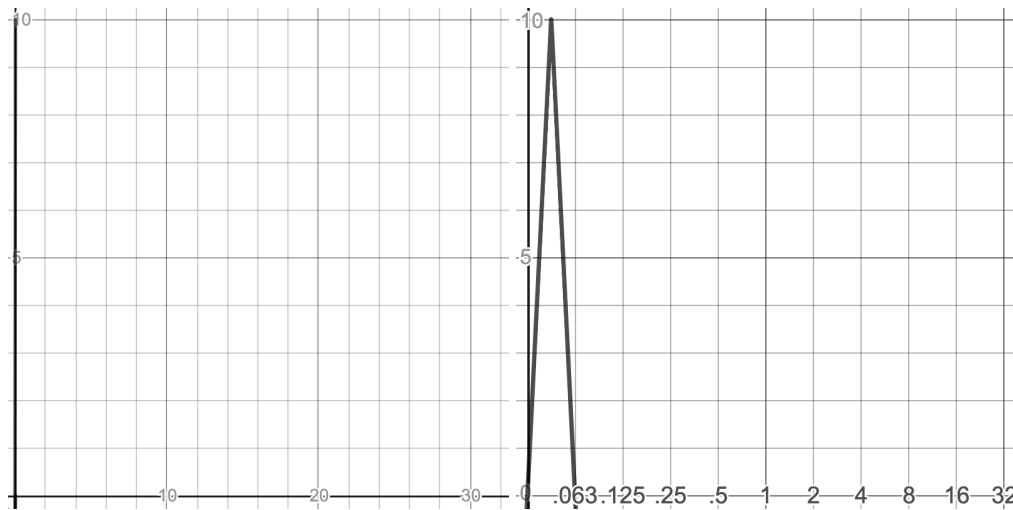
path which would be traversed on a linear scale graph. This causes us to sacrifice accuracy on the line that connects our two **Points** (remember, our midpoint should be [16,5] but is being [1,5]) but it makes it easier to express intent.

This is also important because each **Envelope** has a **curve** parameter that can be adjusted. If we wanted the value between our two points to increase quickly at first, then slowly toward the end we could set our **curve** to -1.0 , which would create a logarithmic path between our two **Points**. If we were to graph this function accurately on our logarithmic scale and a linear scale, we'd end up with the graphs from Proposal 2, and while our resulting graph would be accurate our intent wouldn't be correctly displayed.

There is one more caveat here. Each **Envelope** view shows the animated live paths of all currently active **Voices** as they traverse the **Envelope**. In this case, accuracy is more important than intent, and these animations use the correct values for the x-axis. This means that for our **Envelope** from the examples, the path the **Voice** animations will trace out would be the path from Proposal 1.

6.2.5 But Why?

At this point you may wonder why we should even bother with the logarithmic scale in the first place. What benefit do we get by using a scale like this? Consider another set of examples. Here we want to create a snappy, percussive **Envelope** for our **Amplifier**, with a fast rise and fall. We can describe our desired behavior with three **Points**: [0,0], [0.03125, 10], and [0.0625, 0].



The first graph above shows this on a linear scale. Note that it's almost impossible to differentiate between the y-axis and our **Envelope**'s graph. Compare this with the graph above on the right where we can immediately understand the intention of our **Envelope**. The logarithmic scale gives us higher resolution at the lower end of the x-axis at the cost of resolution on the higher end, and for synthesis purposes it's much more useful to have it this way.

6.2.6 Module Specific

The top area of the **Modulation** section is dedicated to the selected module. The primary parameter from the **Algorithm** section is duplicated here, as well as any other parameters associated with the module (**oscillator sync**, **filter resonance**, etc.).

6.2.7 Modulation

The **Modulation** section allows the selection of a **Modulation Source** which affects the given module independently of the **Envelope**. See the **Sources** section for a description of available **Sources**. The **multiplier** goes from -100% to 100%. At 0% the **Source** has no effect. At 100% the **Source** spans the full range of the **Envelope** (either -5 to 5 or 0 to 10, depending). At -100% the range is inverted (i.e., goes from 5 to -5 or -10 to 0, depending).

6.2.8 Curve

This controls the interpolation between **Points** in an **Envelope**. The range goes from -1.0 (logarithmic) to 1.0 (exponential) with 0.0 being linear.

6.2.9 Random Rates

Each **Envelope** has an independent set of **Random Sources**.

6.2.9.1 Stepped Rate

This controls the rate at which the **Envelope's stepped random source** changes. The range is nonlinear and goes from 10ms to 10s with 1s as the center value. This value is also scaled by the global **time scaling** parameter.

6.2.9.2 Smoothed Rate

This controls the rate at which the **Envelope's smoothed random source** changes. The range is nonlinear and goes from 10ms to 10s with 1s as the center value. This value is also scaled by the global **time scaling** parameter.

6.2.10 Envelope Drawing

The currently selected **Envelope's** state is displayed here. Each circle represents one of the **Envelope's Points**. **Points** are red except for the currently selected **Point** which is white.

To add a new **Point**, double-click in the desired location in the grid. A new **Point** will be created and selected as the current **Point**.

To select an existing **Point** for editing, click it once. The **Point** will change to white and the **Point** display below will change to reflect this. The **time** and **level** values of a **Point** can be changed by dragging it around the **Envelope** drawing area with some limitations:

1. The first **Point** in an **Envelope** must occur at the minimum **time** value (0s) and cannot be moved horizontally.
2. A **Point** cannot be moved before or after its neighbors.
3. A **Point** can be moved to the *same* **time** as its neighbors - this creates an immediate jump to the new **level** without any interpolation.

4. The last **Point** in an **Envelope** cannot exceed the maximum **time** value (32s without **time scaling**).

To remove an existing **Point**, double-click it and the **Point** will disappear. If the **Point** has any neighbors they will be automatically connected to each other. The first **Point** in every envelope cannot be deleted.

6.2.11 Copy/Paste

To the right of the **Envelope**'s title are the **copy** and **paste** buttons. Both **copy** and **paste** utilize the system's clipboard which makes it possible to create and edit **Envelopes** outside of ID700. This functionality copies all **Envelope** values excluding any module-specific parameters. To **copy** the current **Envelope** to the clipboard click the up arrow button, and to **paste** from the clipboard into the current **Envelope** click the down arrow button.

6.2.12 Envelope Randomization

To the right of the **copy** and **paste** buttons is the **randomize envelope** button. Clicking on the dice will randomize the current **Envelope**.

6.2.13 Point

This displays the currently selected **Point**. Clicking the left/right arrows will select the next/previous **Point**.

6.2.13.1 Time

This is the **time** value (x-axis) for the currently selected **Point**. It cannot be changed for **Point 1**. For all other **Points** this value is constrained by the surrounding **Points**, e.g., **Point 3**'s **time** can never be higher than that of **Point 4**. The slider is exponentially scaled from 0-32s.

6.2.13.2 Level

This is the **level** (y-axis) an **Envelope** will reach at the given **time**. The range is linear and is either unipolar (0 to 10) or bipolar (-5 to 5) depending on the module.

6.2.14 Point Modulation

This is an additional **Modulation Source** that applies only to the selected **Point**.

6.2.14.1 Source

See the **Sources** section for a description of available **Sources**.

6.2.14.2 Mult

This is the **multiplier** for the selected **Point**'s **Source**. At 0% the **Source** has no effect. At 100% the **Source** spans the full range of the **Envelope** (either -5 to 5 or 0 to 10 depending). At -100% the range spans the *negative* full range of the **Envelope** (either 5 to -5 or -10 to 0).

6.2.15 Conditioned Actions

Once a **Point** is reached the **Conditioned Actions** of that **Point** are used to calculate where to go next. The possible settings are:

6.2.15.1 None

The **Envelope's** path continues uninterrupted from this **Point** to the next one.

6.2.15.2 Pause If Key Down

If there's a currently playing MIDI note the **Envelope** pauses here. When the MIDI note ends the **Envelope** continues, similar to the sustain section of a classical ADSR.

If the currently playing note has already ended, this setting has no effect and the **Envelope's** path continues along uninterrupted.

6.2.15.3 Stop If Key Up

If there's a currently playing MIDI note the **Envelope** continues on uninterrupted to the next **Point**.

If the currently playing note has already ended then the **Envelope's** path terminates here and will not progress to the next **Point**.

6.2.15.4 Jump To Point

The **Envelope's** path immediately jumps to the selected **Point**. This is repeated an arbitrary number of times.

6.2.15.4.1 Always

The **Envelope's** path will always jump to the selected **Point**.

6.2.15.4.2 N Times

The **Envelope's** path will jump to the selected **Point** **N** times. After the last jump the **Envelope's** path will continue to the next **Point**.

6.2.15.4.3 Rand(N) Times

The **Envelope's** path will jump to the selected **Point** **M** times, where **M** is a random number between 1 and **N** inclusive. After the last jump the **Envelope's** path will continue to the next **Point**.

6.2.15.5 Jump To Point If Key Down

If there's a currently playing note the **Envelope's** path will jump to the selected **Point**.

If there's not a currently playing note the **Envelope's** path will continue along to the next **Point**.

6.2.15.6 Jump To Point If Key Up

If there's a currently playing note the **Envelope's** path will continue along to the next **Point**.

If there's not a currently playing note the **Envelope's** path will jump to the selected **Point**.

6.2.16 Sources

Each **Envelope** can be modulated by a variety of internal and external **Sources**.

6.2.16.1 Voice Sources

Each of these **Sources** can be unique for each **Voice**.

6.2.16.1.1 Pitch Bend

This is the bipolar pitch bend (MIDI) or glide (MPE) value which goes from -5 to 5 with 0 representing no pitch bend. **Pitch Bend** is only unique per **Voice** in MPE mode.

6.2.16.1.2 Note On Velocity

This is the note on velocity (MIDI) or strike (MPE) value from the currently playing note. **Note On Velocity** is always unique per **Voice**.

6.2.16.1.3 Note Off Velocity

This is the note off velocity (MIDI) or lift (MPE) value from the currently playing note. Note that this value isn't available until the MIDI/MPE note has stopped. **Note Off Velocity** is always unique per **Voice**.

6.2.16.1.4 Timbre

This is the timbre (MPE) or brightness (MIDI CC74) value of the currently playing note. **Timbre** is only unique per **Voice** in MPE mode.

6.2.16.1.5 Pressure

This is the pressure (MPE) or aftertouch (MIDI) value of the currently playing note. **Pressure** is always unique per **Voice** in MPE mode, and may be unique per **Voice** in MIDI mode depending on your controller/host combination.

6.2.16.1.6 Pitch

Pitch is based on the currently playing MIDI note. **Pitch** uses the **keyboard range** setting to determine its value. It increases linearly with each successive interval adding a fixed amount. For example, if the **keyboard range** is set to C3-C5 then the **Pitch Source** for C3 will be 0, C4 will be 50%, and C5 will be 100%. Any notes below or above the **keyboard range** will be clipped to the bottom/top of the range. Note that the **Pitch Source** is derived from equal temperament tuning and is independent of any currently selected **Tuning**.

6.2.16.1.7 Frequency

Frequency is based on the currently playing MIDI note. **Frequency** uses the **keyboard range setting** to determine its value. It increases exponentially, doubling each octave, and only becomes significant at very high pitches. For example, if the **keyboard range** is set to C3-C5 then the **Frequency Source** for C3 will be 0, C4 will be 33%, and C5 will be 100%. Any notes below or above the **keyboard range** will be clipped to the bottom/top of the range.

6.2.16.2 Envelope Sources

These **Sources** are unique per *Voice and* per *Envelope*.

6.2.16.2.1 Stepped Random

The **Stepped Random Source** generates a new random value at an arbitrary frequency. The frequency can be adjusted from 10ms to 10s. The random generator has a flat distribution.

6.2.16.2.2 Smoothed Random

The **Smoothed Random Source** is identical to the **Stepped Random Source** except it smoothly interpolates between values.

6.2.16.3 Global Sources

These **Sources** are shared globally by all *Voices*. All **Global Sources** support 14-bit CC messages.

6.2.16.3.1 Mod Wheel

This is the MIDI Modulation Wheel value (CC #1).

6.2.16.3.2 Breath

This is the MIDI Breath Controller value (CC #2).

6.2.16.3.3 Pedal

This is the MIDI Foot Controller value (CC #4).

6.2.16.3.4 Expression

This is the MIDI Expression value (CC #11).

7 Info View

This section contains various information about ID700, and is accessed by clicking the logo in the navigation bar.

7.1 About

This section contains the credits for ID700. You should read them.

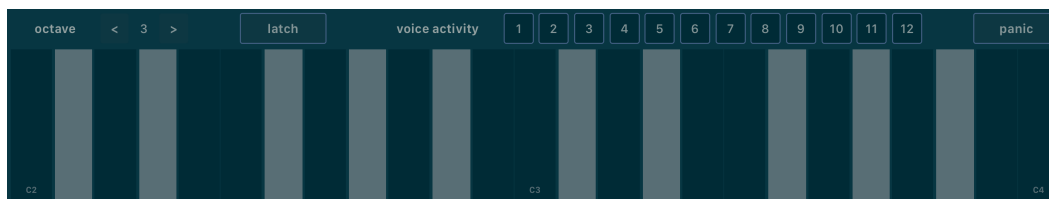
7.2 Documentation

This contains a copy the documentation you're currently reading.

7.3 Debugger

This section allows direct access to the currently loaded **Preset**'s data structure. You can manually examine and set parameters here, but since this bypasses all of the UI functionality you could easily set invalid data and cause a crash. Caveat Utilitor!

8 Controller View



ID700 includes a built-in **Controller**. It supports MIDI and MPE, and the behavior changes based on the current MIDI mode.

8.1 Octave

Each C note is labelled with the octave (C1, C2, etc.). You can shift the range by using the < and > buttons on either side of the displayed **octave** number.

8.2 Latch

When **latch** is enabled all notes played on the **Controller** will sustain until **latch** is disabled. **latch** does not affect notes from external MIDI sources.

8.3 Voice Activity

ID700 has twelve **Voices**. This section shows the current state of each **Voice**. The **Voice**'s square will light up green when a **Voice** is active, and go back to blue when it is inactive.

8.4 Panic

Pressing the **panic** button will stop any currently playing notes from internal or external MIDI/MPE sources. This functionality is also accessible via MIDI, see the MIDI section for details.

8.5 Keyboard

This is a standard two-octave keyboard which includes some extra MPE functionality.

8.5.1 Pitch Bend

After pressing a note you can slide your finger horizontally to add **Pitch Bend**. In MIDI mode this is standard (global) pitch bend, and in MPE mode this is transmitted per-note.

8.5.2 Timbre

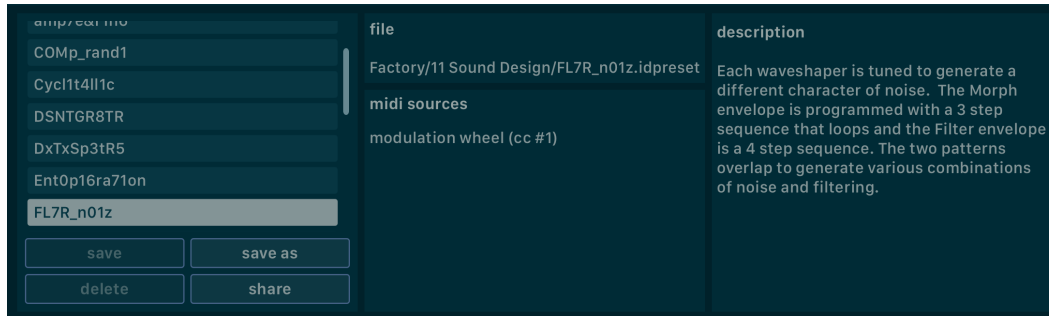
While holding a note you can slide your finger vertically to control **Timbre**. In MIDI mode this is a single global value, and in MPE mode this is transmitted per-note.

8.6 MIDI Out

The following MIDI events are sent from the **Controller**:

- Note On
- Note Off
- Timbre
- Pitch Bend

9 Preset View



Clicking the **Preset** item in the header will open the **Preset** view. Here you can perform **Preset**-related librarian functions.

9.1 .idpreset Files

ID700 Presets have the **.idpreset** file extension. Double-clicking on an **.idpreset** file will import it into the **User/** directory. All **Presets** live in a central directory that is shared between the various formats of the plugin.

9.2 .idpresetpack Files

ID700 can import and export multiple **Preset** files at the same time. These files are called **Preset Packs** and have the **.idpresetpack** extension. Internally these are zip files which are extracted to the **User Presets** directory.

9.3 Browser

The browser section allows you to browse for and perform basic file management operations for **Presets**. All **Presets** live in a central directory that is shared between all the various formats of the plugin.

9.3.1 Save

Clicking the **save** button will save the current **Preset** to disk.

9.3.2 Save As

Clicking the **save as** button will bring up a dialog to specify a new name for the current **Preset**. If you wish to put the file in a subdirectory you must specify it as part of the name, e.g., **My New Folder/My New Preset** (nonexistent directories will automatically be created).

9.3.3 Delete

Clicking the **delete** button will delete the current **Preset** file from disk. Once you click **ok** in the dialog, the file is removed from the file system and your current **Preset** will now be

the next closest **Preset** in the same directory. If your **Preset** file was in a directory which is now empty the directory will also be removed.

9.3.4 Share

Clicking the **share** button will open the native file sharing dialog. This functionality is only available on iOS.

9.3.5 Import

Clicking the **import** button will open the native file dialog so an **.idpresetpack** file can be selected. Once a file is selected the **Preset Pack** will be extracted into the **User Presets** directory.

9.3.6 Export

Clicking the **export** button will export the currently displayed directory as a **Preset Pack**. A dialog prompts for the name of the **Preset Pack**—this is the name of the directory the **Preset** files will be extracted into. Once you click **ok** a native file dialog will open allowing you to select the path to export to.

9.4 File

This section displays the currently selected **Preset** file.

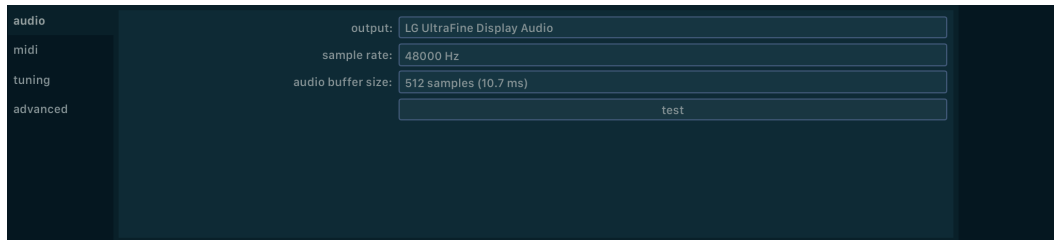
9.5 MIDI Sources

This section displays the **MIDI Sources** to which the currently selected **Preset** responds.

9.6 Description

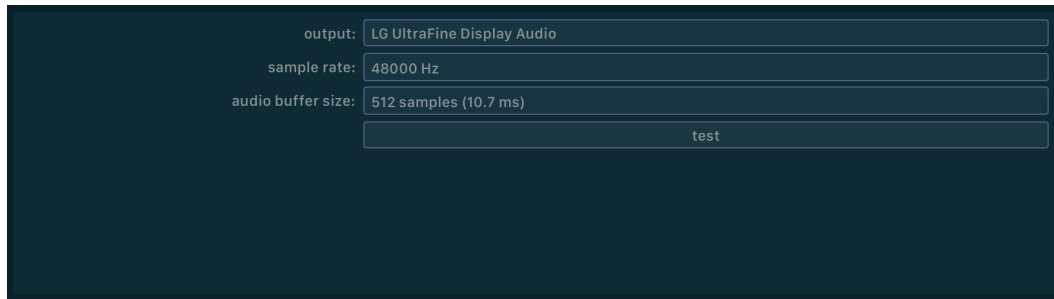
This section displays the optional **description** field of the currently selected **Preset**. Clicking this area allows edits to this text. To save edits, click the **save** button.

10 Settings View



The **Settings** view displays a tabbed menu on the left-hand side. Clicking a given option opens it up in the center panel. All settings are globally set and are not stored with a **Preset**. Some of these settings may not be available depending on your host/platform combination.

10.1 Audio



This view contains settings related to audio input/output.

10.1.1 Output

This allows you to manually select the specific audio device to send output to. The default value should be fine for most use cases.

10.1.2 Sample Rate

This allows you to manually set the sample rate. The default value should be fine for most use cases.

10.1.3 Audio Buffer Size

This allows you to manually set the buffer size. Smaller buffers means lower latency but can result in skipping/artifacts. Larger buffers resulting in fewer artifacts but higher latency. The default value should be fine for most use cases.

10.1.4 Test

Press this button to hear a brief sine wave output. This allows you to test your audio configuration.

10.2 MIDI



This view contains settings related to MIDI/MPE.

10.2.1 MIDI Mode

This allows you to choose to use either MIDI or MPE. Changing this also changes the behavior of the **Controller** (see the **Controller** section for more info). Note that if you are running ID700 as a plugin you'll need to verify that your host supports MPE.

10.2.2 MIDI Channel

This allows you to select the MIDI channel to use. This option is only displayed if MIDI mode is selected. MIDI data not destined for this channel (or channels) will be ignored.

10.2.3 MPE Mode

This allows you to explicitly set the MPE zone to **low** or **high**. This option is only displayed if MPE mode is selected. While in MPE mode ID700 will respond to MIDI data on all channels regardless of the configured **voice count** (this is a workaround for some buggy MPE hardware implementations).

10.2.4 Pitch Bend Range

This allows you to set the pitch bend range from [0-48] semitones. This is the farthest pitch bend can go in either direction, so a range of 12 is displayed as “-/+ 12 semitones”.

10.2.5 Slider CC

With this setting enabled ID700 will transmit CC data for all sliders that respond to MIDI (see Control Numbers for the complete list).

10.2.6 Keyboard Range

The **keyboard range** is used to compute the value of the **Pitch** and **Frequency Sources**. It defines the range that both of these **Sources** use for their minimum/maximum values, see the **Sources** section for more information.

10.2.7 Active MIDI Inputs

This allows you to select a specific MIDI input/device.

10.2.8 MIDI Output

This allows you to select a specific MIDI output/device. This device will receive:

1. All incoming MIDI data
2. MIDI events generated by the **Controller**
3. CC messages from sliders if **slider cc** is enabled

MIDI Output is currently disabled on MacOS pending a bug fix from Apple.

10.2.9 Bluetooth Midi

This opens the system Bluetooth MIDI dialog so ID700 can connect to Bluetooth MIDI devices.

10.3 Tunings



	file	pitches
install tunings	Factory/12_equal_temperament.scl	100.0
		200.0
	scale length	300.0
		400.0
12_equal_temperament	12	500.0
		600.0
	description	700.0
	Twelve-tone equal temperament	800.0
		900.0
		1000.0
		1100.0
		2/1

ID700 supports alternate tunings via the Scala scale file format and includes the entire Scale Archive. **Tunings** are stored in the global settings independently of **Presets**.

10.3.1 Import

To import custom Tuning files just double-click on them. ID700 registers itself as a viewer of **.scl** files and will automatically copy the file into the **User** directory of **Tunings**.

10.3.2 Install

Clicking this button will install the **Tunings** archive. This is quite large and can take up to 30 seconds on older devices. Note this button will only be visible if the **Tunings** archive hasn't been installed.

10.3.3 Browser

This allows you to browse the installed **Tuning** files. Clicking a file will select it as the new active **Tuning**.

10.3.4 File

This is the relative path to the currently selected **Tuning** file.

10.3.5 Scale Length

This is the scale length of the currently selected **Tuning** file. This data comes from the scale length line of the **Tuning** file.

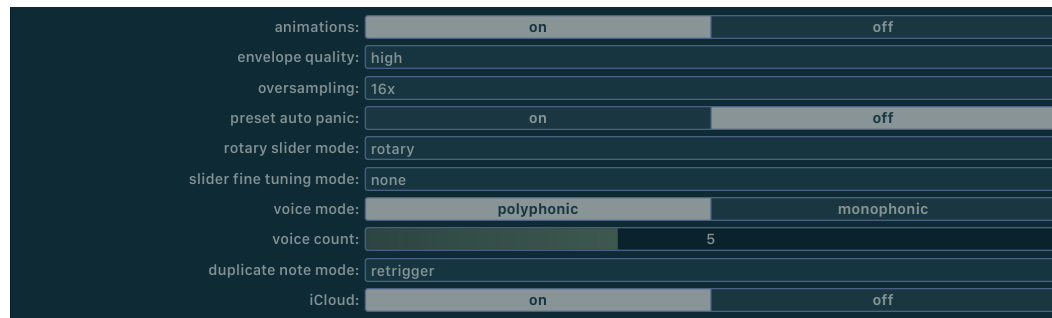
10.3.6 Description

This is the description (if any) of the currently selected **Tuning** file. This data comes from the description of the **Tuning** file.

10.3.7 Pitches

These are the pitch values from the currently selected **Tuning** file. This data comes from the pitch data of the **Tuning** file.

10.4 Advanced



This section contains various uncommonly used settings.

10.5 Animations

ID700's animations are lightweight but do cost CPU time. If performance is an issue you can disable animations here. The default value is **on**.

10.6 Envelope Quality

ID700's **Envelopes** can run at various quality levels. Higher quality allows for snappier **Envelopes** but costs more CPU time. The following values are available:

- **high**—**Envelopes** are calculated at the full sample rate
- **medium**—**Envelopes** are calculated at one half of the sample rate and interpolated up
- **low**—**Envelopes** are calculated at one quarter of the sample rate and interpolated up

Note that some percussive factory **Presets** depend on the **High Envelope** quality and may not sound correct with other values.

10.7 Oversampling

ID700's **Algorithms** support oversampling. Oversampling is a DSP technique which improves the resolution and signal-to-noise ratio of a signal at the cost of more processing time. This technique can reduce the noise and aliasing which are inherent to digital FM and waveshaping

synthesis, and is especially useful at the higher end of the frequency spectrum. The following values are available:

- **none**—Algorithms are calculated at the full sample rate
- **2x**—Algorithms are calculated at two times the original sample rate, filtered, and downsampled back to the original rate
- **4x**—Algorithms are calculated at four times the original sample rate, filtered, and downsampled back to the original rate
- **8x**—Algorithms are calculated at eight times the original sample rate, filtered, and downsampled back to the original rate
- **16x**—Algorithms are calculated at sixteen times the original sample rate, filtered, and downsampled back to the original rate

*Note: There's no guarantee that **Oversampling** will be performant on your hardware. Higher **Oversampling** values will use more CPU, may reduce effective polyphony, and may require a corresponding increase of the Audio Buffer Size. YMMV.*

10.8 Voice Mode

This setting controls how ID700 reacts to incoming MIDI notes. Valid values are **Polyphonic** and **Monophonic**.

10.8.1 Polyphonic

In **Polyphonic** mode each incoming MIDI note is assigned to an available **Voice**. The **Voice Count** and **Duplicate Note Mode** settings control this behavior.

10.8.1.1 Voice Count

This setting controls the number of **Voices** available in ID700. The valid range is [1,12] with a default value of 12.

10.8.1.2 Duplicate Note Mode

This setting controls how ID700 behaves when an incoming MIDI note duplicates the note of an active **Voice**. The following modes are available:

- **retrigger**—ID700 will retrigger the incoming note on the active **Voice**
- **reset**—ID700 will reset the active **Voice** and trigger the incoming note on the next available **Voice**
- **rotate**—ID700 will start at the first **Voice**. Each subsequent note will use the next highest **Voice**. Once the highest available **Voice** is used the next note will wrap around to **Voice 1**. If the next **Voice** is active it will be reset before the new note is triggered.

10.8.2 Monophonic

In **Monophonic** mode ID700 has only one **Voice**. An initial note will trigger the **Voice's Envelopes**, and the **Envelopes** will continue to progress until *all* notes are released. When multiple sustained notes are present the **Note Priority** setting determines which note is actually played.

10.8.2.1 Note Priority

This setting determines which note ID700 will treat as active when more than one note is sustained. The active note provides the following information:

- `pitch bend`
- `note on velocity`
- `note off velocity`
- `timbre`
- `pressure`
- `pitch`
- `frequency`

ID700 will update all of the above when the set of sustained notes changes. The following modes are available:

- `first note`—ID700 will select the earliest played note
- `last note`—ID700 will select the most recently played note
- `lowest note`—ID700 will select the note with the lowest MIDI note number
- `highest note`—ID700 will select the note with the highest MIDI note number

10.9 Preset Auto Panic

This setting controls how ID700 behaves when changing **Presets**.

- `on`—ID700's **panic** function is called before the new **Preset** is loaded, which causes all active **Voices** to be stopped
- `off`—any active **Voices** will continue to sound while changing **Presets**

The default value is `off`.

10.10 Rotary Slider Mode

This setting controls how ID700's rotary sliders respond to input. The following modes are available:

- `rotary`—dragging the mouse in a circular motion changes the value
- `horizontal`—dragging the mouse left-to-right changes the value
- `vertical`—dragging the mouse down-to-up changes the value
- `horizontal and vertical`—dragging the mouse left-to-right or down-to-up changes the value

`rotary slider mode` controls the sliders for `algorithm`, `fine tuning`, `time scaling`, and `main out`.

10.11 Slider Fine Tuning Mode

This setting controls the fine control of ID700's horizontal sliders.

- `none`—sliders only respond to horizontal drags
- `vertical`—dragging vertically from the slider causes a fine (small) adjustment

`slider fine tuning mode` controls all horizontal sliders except for any **settings**.

10.12 iCloud

This setting controls where **User Presets**, **User Waveshapes**, and **User Tunings** are stored.

- **on**—**User Presets**, **User Waveshapes**, and **User Tunings** are stored in iCloud.
- **off**—**User Presets**, **User Waveshapes**, and **User Tunings** are stored locally

A cloud icon will display next to the **User** directories stored in iCloud.

On MacOS this setting is enabled by default if iCloud is available. On iOS this setting is enabled by default and cannot be changed.

When this setting changes, ID700 will copy all of the aforementioned files from the old location to the new location. If a file with the same name already exists in the new location it will be overwritten.

11 MIDI Implementation

11.1 Program Change

ID700 responds to MIDI program change messages. Attempting to access a nonexistent **Bank / Preset** combination will silently fail. **Factory Presets** are indexed differently than **User Presets**.

11.1.1 Factory Presets

Factory Presets have stable program numbers (i.e., **Bank 0, Program 0** will always be **De Fault**). **Factory Presets** start at **Bank 0**. The **Factory Preset** section lists the available **Banks** and **Programs**.

11.1.2 User Presets

User Presets do not have stable program numbers (i.e., **Bank 16, Program 0** will be the first **User Preset** which could change when new **User Presets** are added or deleted). They are accessible starting at **Bank 16**.

11.2 Thru

ID700 sends a copy of all incoming MIDI data to the selected MIDI output.

11.3 Continuous Parameters

ID700 supports high resolution (14-bit) values for all **Continuous** parameters *except* **Timbre** (blame the MIDI spec).

11.4 On / Off Parameters

On / Off parameters are **Off** when the **CC** value is **[0,63]** and **On** when **[64,127]**. **Randomize Preset**, **All Sound Off**, and **Reset All Controllers** are momentary switches and will not retrigger until the **CC** value drops below **64**.

11.5 Control Numbers

ID700 generates and responds to the following MIDI control numbers.

Control #	Parameter	Generated?	Type	Notes
0	Bank Select MSB	No	Continuous	See Program Change
1	Modulation Wheel MSB	No	Continuous	Modulation Source
2	Breath MSB	No	Continuous	Modulation Source
4	Pedal MSB	No	Continuous	Modulation Source
7	Amplifier MSB	Yes	Continuous	
10	Location MSB	Yes	Continuous	
10	Location MSB	Yes	Continuous	
11	Expression MSB	No	Continuous	Modulation Source
12	Algorithm MSB	Yes	Continuous	
13	Time Scaling MSB	Yes	Continuous	
14	Fine Tuning MSB	Yes	Continuous	
15	Main Out MSB	Yes	Continuous	
16	Oscillator 1 MSB	Yes	Continuous	
17	Oscillator 2 MSB	Yes	Continuous	
18	Oscillator 3 MSB	Yes	Continuous	
19	Oscillator 4 MSB	Yes	Continuous	
20	Index 1 MSB	Yes	Continuous	
21	Index 2 MSB	Yes	Continuous	
22	Index 3 MSB	Yes	Continuous	
23	Index 4 MSB	Yes	Continuous	
24	Index 5 MSB	Yes	Continuous	
25	Index 6 MSB	Yes	Continuous	
26	Morph MSB	Yes	Continuous	
27	Filter Frequency MSB	Yes	Continuous	
28	Wave A Odd/Even MSB	Yes	Continuous	
29	Wave B Odd/Even MSB	Yes	Continuous	
30	Wave A Tilt MSB	Yes	Continuous	
31	Wave B Tilt MSB	Yes	Continuous	
32	Bank Select LSB	No	Continuous	Program Change
33	Modulation Wheel LSB	No	Continuous	Modulation Source
34	Breath LSB	No	Continuous	Modulation Source
36	Pedal LSB	No	Continuous	Modulation Source
39	Amplifier LSB	Yes	Continuous	
42	Location LSB	Yes	Continuous	
43	Expression LSB	No	Continuous	Modulation Source
44	Algorithm LSB	Yes	Continuous	

Control #	Parameter	Generated?	Type	Notes
45	Time Scaling LSB	Yes	Continuous	
46	Fine Tuning LSB	Yes	Continuous	
47	Main Out LSB	Yes	Continuous	
48	Oscillator 1 LSB	Yes	Continuous	
49	Oscillator 2 LSB	Yes	Continuous	
50	Oscillator 3 LSB	Yes	Continuous	
51	Oscillator 4 LSB	Yes	Continuous	
52	Index 1 LSB	Yes	Continuous	
53	Index 2 LSB	Yes	Continuous	
54	Index 3 LSB	Yes	Continuous	
55	Index 4 LSB	Yes	Continuous	
56	Index 5 LSB	Yes	Continuous	
57	Index 6 LSB	Yes	Continuous	
58	Morph LSB	Yes	Continuous	
59	Filter Frequency LSB	Yes	Continuous	
60	Wave A Odd/Even LSB	Yes	Continuous	
61	Wave B Odd/Even LSB	Yes	Continuous	
62	Wave A Tilt LSB	Yes	Continuous	
63	Wave B Tilt LSB	Yes	Continuous	
64	Sustain Pedal	No	On / Off	
74	Timbre	Yes	Continuous	Modulation Source
119	Randomize Preset	No	On / Off	Identical to pressing the Randomize button
120	All Sound Off	No	On / Off	Identical to pressing the Panic button
121	Reset All Controllers	No	On / Off	Reloads the current Preset

12 .idwave File Format

ID700 reads and writes **Waveshape** information from **.idwave** files. Internally these are simple XML files. The **.idwave** file format contains level information for the first 33 **Harmonics** as well as some file metadata.

Here's an example of an **.idwave** file:

```
<Waveshape file="Factory/blue/blue 3.idwave">
  <Harmonics>
    <Harmonic order="0" level="-0.1500000357627869"/>
    <Harmonic order="1" level="0.1699999570846558"/>
    <Harmonic order="2" level="-0.1700000166893005"/>
    <Harmonic order="3" level="0.1699999570846558"/>
    <Harmonic order="4" level="0.0"/>
    <Harmonic order="5" level="-0.1700000166893005"/>
    <Harmonic order="6" level="0.0"/>
    <Harmonic order="7" level="0.1699999570846558"/>
    <Harmonic order="8" level="0.0"/>
    <Harmonic order="9" level="0.0"/>
    <Harmonic order="10" level="0.0"/>
    <Harmonic order="11" level="0.1699999570846558"/>
    <Harmonic order="12" level="0.0"/>
    <Harmonic order="13" level="-0.1700000166893005"/>
    <Harmonic order="14" level="0.0"/>
    <Harmonic order="15" level="0.0"/>
    <Harmonic order="16" level="0.0"/>
    <Harmonic order="17" level="0.1699999570846558"/>
    <Harmonic order="18" level="0.0"/>
    <Harmonic order="19" level="-0.1700000166893005"/>
    <Harmonic order="20" level="0.0"/>
    <Harmonic order="21" level="0.0"/>
    <Harmonic order="22" level="0.0"/>
    <Harmonic order="23" level="0.1699999570846558"/>
    <Harmonic order="24" level="0.0"/>
    <Harmonic order="25" level="0.0"/>
    <Harmonic order="26" level="0.0"/>
    <Harmonic order="27" level="0.0"/>
    <Harmonic order="28" level="0.0"/>
    <Harmonic order="29" level="-0.1700000166893005"/>
    <Harmonic order="30" level="0.0"/>
    <Harmonic order="31" level="0.1699999570846558"/>
    <Harmonic order="32" level="0.0"/>
  </Harmonics>
</Waveshape>
```


12.1 <Waveshape>

The <Waveshape> element is the main container. It requires a `file` attribute which is the expected path to the `.idwave` file relative to the `Waveshapes` directory. It should contain only a single <Harmonics> element.

12.2 <Harmonics>

The <Harmonics> element contains 33 child <Harmonic> elements.

12.3 <Harmonic>

Each <Harmonic> has two attributes, `order` and `level`. `order` indicates which Harmonic the element represents and must be in the range [0,32]. `level` indicates the level of the Harmonic and must be in the range [-1, 1]. A negative level inverts the phase of the Harmonic.

12.4 DTD

Here's a DTD for `.idwave` files:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE Waveshape [
  <!ELEMENT Waveshape (Harmonics)>
  <!ATTLIST Waveshape file CDATA #REQUIRED>
  <!ELEMENT Harmonics (Harmonic+)>
  <!ELEMENT Harmonic EMPTY >
  <!ATTLIST Harmonic
    order
      (0|1|2|3|4|5|6|7|8|9|10|11|12|13|14|15|16|17|18|19|20|21|22|23|24|25|26|27|28|29|30|31|32)
      #REQUIRED
    level CDATA #IMPLIED>
]>
```

13 Envelope Format

ID700 reads and writes **Envelope** information as XML data.

Here's an example of an **Envelope**:

```
<Envelope id="2" generalSource="0" generalSourceMult="0.0" curve="0.0"
    smoothedRandomRate="1.0" steppedRandomRate="1.0" active="1">
  <Segment time="0.0" level="-0.79" conditionedActionType="0" source="0"
    sourceMult="0.0" conditionedActionIterator="0" conditionedActionTarget="-1"/>
  <Segment time="0.000637692690361291" level="2.66" conditionedActionTarget="-1"
    conditionedActionIterator="0" sourceMult="0.0" source="0"/>
  <Segment time="0.1677334755659103" level="-1.58" conditionedActionTarget="-1"
    conditionedActionIterator="0" sourceMult="0.0" source="0"/>
</Envelope>
```

13.1 <Envelope>

The **<Envelope>** element is the main container. It contains the following attributes

13.1.1 id

The **id** attribute declares which **Envelope** this structure refers to. ID700 uses **id** when copying and pasting **Envelopes** to determine the correct **level** range for all **Segments**. ID700 will automatically convert the ranges of pasted **Envelopes** based on the source and target **id**. The valid **Segment** range for a bipolar **Envelope** is [-5,5], while a unipolar **Envelope** expects [0-10]. The following chart lists the possible **id** values:

id	Name	Bipolar
0	Oscillator 1	yes
1	Oscillator 2	yes
2	Oscillator 3	yes
3	Oscillator 4	yes
4	Filter	no
5	Morph	yes
6	Location	yes
7	Index 1	no
8	Index 2	no
9	Index 3	no
10	Index 4	no
11	Index 5	no
12	Index 6	no
13	Amplifier	no

13.1.2 curve

The **curve** attribute determines the curve the **Envelope** uses to traverse between **Points**. The valid range is [-1, 1] where -1 is logarithmic, 0 is straight, and 1 is exponential.

13.1.3 `generalSource`

The `generalSource` attribute selects the `Envelope's Modulation Source`. The following chart lists the possible `generalSource` values:

<code>generalSource</code>	Name
0	None
1	Pitch Bend
2	Note On Velocity
3	Note Off Velocity
4	Timbre
5	Pressure
6	Pitch
7	Frequency
8	Smoothed Random
9	Stepped Random
10	Modulation Wheel
11	Breath
12	Pedal
13	Expression

13.1.4 `generalSourceMult`

The `generalSourceMult` attribute determines the multiplier applied to the `generalSource`. The valid range is `[-1,1]` which corresponds to `[-100%,100%]`.

13.1.5 `smoothedRandomRate`

The `smoothedRandomRate` attribute determines the period of the `smoothed random source` in seconds. The valid range is `[0.01,10]`.

13.1.6 `steppedRandomRate`

The `steppedRandomRate` attribute determines the period of the `stepped random source` in seconds. The valid range is `[0.01,10]`.

13.1.7 `active`

The `active` attribute describes the currently selected `Segment` in the UI. `active` is zero-indexed so a value of 0 means `Segment 1`.

13.2 `<Segment>`

The `<Segment>` element represents a single `Point` in a `Envelope`. An `<Envelope>` must contain at least one `<Segment>` element.

13.2.1 time

The **time** attribute determines the relative time at which the **Point's level** will be reached in seconds. The valid range is [0,32]. The first <Segment> must have a **time** of 0, and all subsequent <Segments> must be sorted by this value in ascending order.

13.2.2 level

The **level** attribute determines the level an **Envelope** should reach at the given **time**. The valid range for a bipolar **Envelope** is [-5,5], while a unipolar **Envelope** expects [0-10].

13.2.3 source

The **source** attribute selects the **Point's Modulation Source**. See the **generalSource** section above for legal values.

13.2.4 sourceMult

The **sourceMult** attribute determines the multiplier applied to the **source**. The valid range is [-1,1] which corresponds to [-100%,100%].

13.2.5 conditionedActionType

The **conditionedActionType** attribute determines which **Conditioned Action** applies to the **Point**. The following chart lists the legal **conditionedActionType** values:

ID	Conditioned Action Type	Requires Target and Iterator
0	None	No
1	Pause If Down	No
2	Stop If Up	No
3	Jump To	Yes
4	Jump To If Up	Yes
5	Jump To If Down	Yes

13.2.6 conditionedActionIterator

The **conditionedActionIterator** attribute determines how many times a **Point** should jump to its **target**. This attribute is required for certain **conditionedActionType** values, see the **conditionedActionType** chart for more information. The following chart lists the legal **conditionedActionIterator** values:

ID	Iterator
0	Always
1	One Time
2	Two Times
3	Three Times
4	Four Times
5	Five Times

ID	Iterator
6	Six Times
7	Seven Times
8	Eight Times
9	Nine Times
10	Random(Two) Times
11	Random(Three) Times
12	Random(Four) Times
13	Random(Five) Times
14	Random(Six) Times
15	Random(Seven) Times
16	Random(Eight) Times
17	Random(Nine) Times

13.2.7 conditionedActionTarget

The `conditionedActionTarget` attribute determines which Point the current Point should jump to. The value is zero-indexed and must be within the range of current Points.

13.2.8 DTD

Here's an `<Envelope>` DTD:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE Envelope [
  <!ELEMENT Envelope (Segment+)>
  <!ATTLIST Envelope
    id (0|1|2|3|4|5|6|7|8|9|10|11|12|13) #REQUIRED
    curve CDATA #REQUIRED
    generalSource (0|1|2|3|4|5|6|7|8|9|10|11|12|13) #REQUIRED
    generalSourceMult CDATA #REQUIRED
    smoothedRandomRate CDATA #REQUIRED
    steppedRandomRate CDATA #REQUIRED
    active CDATA #IMPLIED>
  <!ELEMENT Segment EMPTY>
  <!ATTLIST Segment
    time CDATA #REQUIRED
    level CDATA #REQUIRED
    source (0|1|2|3|4|5|6|7|8|9|10|11|12|13) #REQUIRED
    sourceMult CDATA #REQUIRED
    conditionedActionType (0|1|2|3|4|5) #REQUIRED
    conditionedActionIterator (0|1|2|3|4|5|6|7|8|9|10|11|12|13|14|15|16|17) #REQUIRED
    conditionedActionTarget CDATA #REQUIRED>
]>
```

14 Factory Presets

ID700 ships with over 220 **Factory Presets** designed by Kurt Kurasaki. To support **Program Change** messages the **Factory Presets** are grouped in **Banks** of up to 128 **Programs**. The following charts show the **Bank** and **Program** numbers for all **Factory Presets**.

14.1 Bank 00 Programs

#	Name	#	Name	#	Name
0	00 De Fault	43	K7aviAltun1ng	86	RND_005
1	arm0ni	44	P1ano1ss1m0	87	RND_006
2	B700_10g1c	45	Xl3ctric Pian0	88	RND_007
3	DRNnn6	46	aRnd5yncs	89	RND_008
4	FM_bl1ne	47	B3ll3 100Ps	90	RND_009
5	S3p4r4710n	48	DtdTdTDt	91	RND_010
6	SEQ_adagio	49	ech0 bonc3	92	RND_011
7	T3mpv5ccrma	50	Exp4ndx	93	RND_012
8	Two40MARF	51	fltr_sek710n	94	RND_013
9	Al14S_bass	52	gnuM0n1x	95	RND_014
10	B0770mf33d	53	m4nu4l 57rumm	96	RND_015
11	B455 67R	54	NZE_L7p	97	RND_016
12	F4T_sl1n3	55	p0lyEasel	98	RND_017
13	fl3ss Db4ss	56	Pl0drxt	99	RND_018
14	h0770wBass	57	PRTL_6473	100	RND_019
15	Nuly b4ss	58	Rtybngr4A4	101	RND_020
16	Squ4r3B455	59	FM_thndrPad	102	RND_021
17	art_sBD	60	k1nesthet1ca	103	RND_022
18	art_sn4R	61	lumina_4bstrct	104	RND_023
19	BD7808	62	mpeTh3r3min	105	RND_024
20	BnG700z	63	P0ly_L34D	106	RND_025
21	ctrl_cl1k	64	press4_lopassgate	107	RND_026
22	cymb15h	65	ALTune_m374l	108	RND_027
23	DR-700	66	Bllegrn6t	109	RND_028
24	DR_SNR3	67	CapanOxidentalis	110	RND_029
25	H1iKik	68	ebutube	111	RND_030
26	Hihz	69	otherbells	112	RND_031
27	IDrkCLP	70	Synclavb4d	113	RND_032
28	kG7ich1t	71	T00bu74r	114	arp3gg10
29	lngg_n0153	72	t1mechx	115	CaZc4de5equence
30	Lxperc	73	TYPE_b3ll	116	D0wn1ng
31	mycVer	74	Xy70rimb4	117	Easel5steps
32	n33dm0r3 c0wp3rx	75	grn_sl7	118	H4lfShift
33	p1nk_HAT	76	humarvxc	119	In5ifths
34	SDR_1fin17y	77	L3_v01	120	M1lls3q
35	Sn4rIDr	78	qlipp3d v0x	121	MAJ_scale
36	ssstick	79	S4r4rr	122	mnrr_Sq
37	Symb71sm	80	v0xM4ths	123	Oct4m0d
38	TRCwb3ll	81	Xephers	124	Tw05ifty
39	AM_f0rt3	82	RND_001	125	up5ive
40	Buchlavier	83	RND_002	126	zCMP_t0wnr4c3r
41	Ch0wnin6for7e	84	RND_003	127	700 Waves
42	DRK_rh0d7	85	RND_004		

14.2 Bank 01 Programs

#	Name	#	Name	#	Name
0	amp7e&r1n6	43	MIDASqkc010	86	S7ubbyFLTR
1	COMp_rand1	44	msblim4710n	87	Sili-Con Ce770
2	Cycl1t4ll1c	45	P1ped1rd6e	88	sm700thAngles
3	DSNTGR8TR	46	p4de	89	SXS4PWND
4	DxTxSp3tR5	47	P7anDsol	90	tunen07une
5	Ent0p16ra71on	48	PhzjunCxl	91	TwoOh8ight
6	FL7R_n01z	49	Prim0rdE4L	92	V1Rcov21
7	mono-DR0n3	50	PRTL_sw33p	93	v3ntas0l0
8	MRT_l00t	51	Qu4r1vs	94	WSBZ_700
9	n30phy7e	52	RED_phzdrn		
10	ohlntqxxq	53	S1n3PWMd		
11	p0pz	54	Sine2Nz7p		
12	r4ndAMp	55	Solmnvtkq		
13	RKadeNzes	56	str1n9_sek710n		
14	sftmc_DR0pl	57	sycgr1ty		
15	six6ix6ix	58	UFnd47tpePD		
16	Sm1th-C0r0n4	59	Vn1vrs4		
17	sttR4ng5	60	W4nd3r3rs		
18	ARP_s1n3	61	xodr4hedron		
19	bsc_5748	62	Yh4rmonic		
20	E0_bl00m	63	Z1n0ida7		
21	f4u5tin4t0	64	4ny 4lgorithm		
22	flux3rs	65	Borg4n		
23	gmm1e	66	Br455 En53mb		
24	jos3nz0	67	Ch3by700th		
25	Pylonj	68	DS lead		
26	Rp3t3r	69	dxPl1c4t10n5		
27	t4ut0l0g	70	Flutw1nd5		
28	Tun3d_NZE	71	FMP1ckVp5		
29	a5cendr	72	h4rDK0de		
30	Ana706ix	73	kalai0rgnII		
31	Beqisjntdd	74	Ken53vix		
32	cmood_TR1NGL	75	LEAD_w4v		
33	D41syd4y	76	LF_Rz3t		
34	Deep 4lgor17hms	77	lopass 4ngles		
35	Dr34m6u1d3	78	LZR_h4rpin9		
36	Dy50nSqu4re	79	mello_7imbre		
37	eniSine	80	ob_PWmd		
38	f07din6m0ds	81	Odd17y		
39	harmnvs	82	ppIDg		
40	ID70ne	83	pr0700		
41	IDeations	84	Qua5ib3ndR		
42	J7un00	85	S7id4rB		

15 About

ID700 version 1.30.0 build 1704

© 2017-2024 Modosc Designs

All Rights Reserved

Web: modo.sc

Email: info@modo.sc

15.1 Credits

Everything by Jonathan Schatz except where noted below

15.1.1 Testers

- Drew Schlesinger
- Warren Lee
- Kurt Kurasaki
- Anonymous
- Anonymous
- Anonymous

15.1.2 Presets

- Kurt Kurasaki

15.1.3 Editor

- Laura Heyenga

15.1.4 Tunings

The factory tunings are all imported from the Scala Scale Archive with permission from Manuel Op de Coul.

15.1.5 Waveshapes

The **red** and **green** factory **Waveshapes** are based on the Buchla 259e's red and green banks created by Don Buchla and Yasi Perera. ID700's versions are not identical but contain similar harmonic content.

15.1.6 Icons

- Joe Encarnacion

15.1.7 Third-Party Software

- Algorithm images from Johan Boberg, originals by Don Buchla and Lynx Crowe
- fontaudio by Michelangelo Nottoli

- `ffGuiAttachments` by Daniel Walz/Foleys Finest Audio UG (haftungsbeschraenkt)
- `ff_meters` by Daniel Walz/Foleys Finest Audio UG (haftungsbeschraenkt)
- `juce-toys` by Jim Credland
- `JUCE`
- `tuning-library` by Paul Walker

15.1.8 Special Thanks

- Laura, Lucas, Isa
- Rick
- Alessandro
- Johan
- Kurt

15.1.9 Extra Special Thanks

- Chris Randall

15.1.10 Shoutouts

- Rob, Jack, Joe and John from Bfamily
- Joel and Darren from Buchla

15.1.10.1 In memoriam Dan and Dean

16 Changelog

[1.30.0] - 2025-10-18

- Fix various view crashes

[1.29.0] - 2024-03-22

- Fix Ableton Live 11 + MacOS 14 view crash
- Fix blank `About/Documentation` links in AUv3 on iOS

[1.28.0] - 2024-03-09

- Fix MacOS crash when changing Algorithm
- Improve display of pitch bend range Midi Settings

[1.27.0] - 2023-12-17

- Upgrade JUCE to v7
- Enable `Preset` sharing in AUv3 on iOS

[1.26.0] - 2023-08-28

- Correctly report MPE support in Ableton Live

[1.25.0] - 2023-07-30

- Add `Oversampling` support

[1.24.0] - 2023-07-09

- Revert “Fix `Timbre (cc #74)` handling in Midi Mode” because it introduced another bug

[1.23.0] - 2023-06-10

- Fix `Timbre (cc #74)` handling in Midi Mode

[1.22.0] - 2023-06-04

- Improve resizing/scaling behavior on iOS for Logic Pro
- Fix `Fine Tuning` rounding bug

[1.21.0] - 2023-02-22

- Add `License` link
- Add `iCloud` setting and initial implementation with support for storing and migrating `User Presets`, `User Waveshapes`, and `User Tunings`

[1.20.0] - 2023-02-16

- Fix toggle button race condition in `Midi/Advanced Settings`
- Fix `iOS Controller` crash

[1.19.0] - 2023-01-22

- Add `Preset Pack` import and export functionality

[1.18.0] - 2022-09-11

- Fix crash when changing sample rates
- New `Algorithm` ui

[1.17.0] - 2022-03-26

- Fix `iOS` crash on receiving `MIDI CC 119 (Randomize Preset)`
- Fix and document `Randomize Preset`, `All Sound Off` and `Reset All Controllers` `CC` behavior
- Fix `MacOS` crash when `Presets` directory isn't writable

[1.16.0] - 2022-02-23

- Add `Monophonic` voice mode
- Add `Note Priority` setting

[1.15.0] - 2022-02-12

- Fix display of `Point Modulation Sources` in the `Preset MIDI Sources` view
- Fix refresh bug in `Preset MIDI Sources` view
- Fix crash on older versions of `MacOS`

[1.14.0] - 2022-01-20

- Add `AUv3` parameter gesture begin/end messages
- Fix `About / Documentation` views when running in `VST3` wrapper

[1.13.1] - 2022-01-08

- Revert default size for `AUM`
- Change default desktop sizing back to `1024x768`

[1.13.0] - 2021-12-22

- Fix scaling of windows $> 1024 \times 768$
- Disable click navigation from `Algorithm` screen when in compact mode
- Revert previous change to `Preset`, `Waveshape`, and `Tuning` installations from `1.12.2`
- Add default size for `AUM`
- Improve `AUv3` resizing behavior

[1.12.2] - 2021-12-19

- Prevent **Preset**, **Waveshape**, and **Tuning** installations from blocking initial startup

[1.12.1] - 2021-12-17

- Fix double-triggering bug in **Controller**

[1.12.0] - 2021-11-21

- Disable MIDI output in desktop builds to workaround **auvaltool** bug
- Add **Preset Randomization** support
- Add **Envelope Randomization** support

[1.11.0] - 2021-10-07

- Add **slider fine tuning** mode setting
- Fix **Waveshape** drawing bug
- Fix iOS crash when using an external audio interface

[1.10.0] - 2021-07-05

- Fix double-click reset of **Envelope** sliders
- Add **midi output** selector to MIDI settings
- Add MIDI output from **Controller**
- Add **slider cc** MIDI setting
- Add MIDI CC output from most sliders

[1.9.0] - 2021-06-14

- Fix sensitivity bug in horizontal sliders
- Make **Harmonic** slider ranges closer to exponential

[1.8.0] - 2021-06-04

- Add MIDI Sustain Pedal support
- Add **Tuning** import
- Add navigation to the correct browser screen after importing a **Tuning**, **Waveshape**, or **Preset**

[1.7.0] - 2021-05-24

- Add **Waveshape** copy/paste support
- Add **Envelope Format** documentation
- Fix AUM session load regression

[1.6.0] - 2021-05-18

- Fix crash in GarageBand export

[1.5.0] - 2021-05-10

- Add vertical fine mode to horizontal sliders
- Add rotary slider mode setting

[1.4.0] - 2021-04-30

- Add rotate to duplicate note modes
- Fix MIDI handling in midi mode to only respond to the configured MIDI channel(s)
- Fix midi sources text editor to be read-only
- Expand pitch bend range to +/- 48 semitones
- Update Presets

[1.3.0] - 2021-04-23

- Update Documentation
- Add MIDI Program Change and Bank Select support
- Fix occasional AUv3 crash on host-initiated Program Change
- Fix broken CC control of algorithm, time scaling, fine tuning, and main out
- Add Program Change access to User Presets

[1.2.0] - 2021-04-19

- Remove JUCE splash screen
- Fix double-clicking on .idpreset and .idwave files in iOS
- Add voice count setting
- Add duplicate note mode setting
- Update documentation
- Increase font weight/size
- Fix smoothed/stepped random ranges
- Add preset auto panic setting
- Fix Location panning math bug

[1.1.0] - 2021-04-14

- Add Changelog
- Prevent iOS from going to sleep while standalone is running
- Update Presets
- Update documentation
- Reduce app bundle size by 33%
- Fix blank About/Documentation links in AUv3
- Add manual Tuning archive installation

[1.0.0] - 2021-04-08

- Initial Release

17 End-User License Agreement (EULA) of ID700

This End-User License Agreement (“EULA”) is a legal agreement between you and **Modosc Designs**

This EULA agreement governs your acquisition and use of our **ID700** software (“Software”) directly from **Modosc Designs** or indirectly through a **Modosc Designs** authorized reseller or distributor (a “Reseller”).

Please read this EULA agreement carefully before completing the installation process and using the **ID700** software. It provides a license to use the **ID700** software and contains warranty information and liability disclaimers.

If you register for a free trial of the **ID700** software, this EULA agreement will also govern that trial. By clicking “accept” or installing and/or using the **ID700** software, you are confirming your acceptance of the Software and agreeing to become bound by the terms of this EULA agreement.

If you are entering into this EULA agreement on behalf of a company or other legal entity, you represent that you have the authority to bind such entity and its affiliates to these terms and conditions. If you do not have such authority or if you do not agree with the terms and conditions of this EULA agreement, do not install or use the Software, and you must not accept this EULA agreement.

This EULA agreement shall apply only to the Software supplied by **Modosc Designs** referred to or described herein. The terms also apply to any **Modosc Designs** updates, supplements, Internet-based services, and support services for the Software, unless other terms accompany those items on delivery. If so, those terms apply. This EULA was created by EULA Template for **ID700**.

License Grant

Modosc Designs hereby grants you a personal, nontransferable, nonexclusive license to use the **ID700** software on your devices in accordance with the terms of this EULA agreement.

You are permitted to load the **ID700** software (for example a PC, laptop, mobile, or tablet) under your control. You are responsible for ensuring your device meets the minimum requirements of the **ID700** software.

You are not permitted to:

- Edit, alter, modify, adapt, translate or otherwise change the whole or any part of the Software nor permit the whole or any part of the Software to be combined with or become incorporated in any other software, nor decompile, disassemble or reverse engineer the Software or attempt to do any such things
- Reproduce, copy, distribute, resell or otherwise use the Software for any commercial purpose
- Allow any third party to use the Software on behalf of or for the benefit of any third party
- Use the Software in any way which breaches any applicable local, national or international law

- use the Software for any purpose that **Modosc Designs** considers is a breach of this EULA agreement

Intellectual Property and Ownership

Modosc Designs shall at all times retain ownership of the Software as originally downloaded by you and all subsequent downloads of the Software by you. The Software (and the copyright, and other intellectual property rights of whatever nature in the Software, including any modifications made thereto) are and shall remain the property of **Modosc Designs**.

Modosc Designs reserves the right to grant licenses to use the Software to third parties.

Termination

This EULA agreement is effective from the date you first use the Software and shall continue until terminated. You may terminate it at any time upon written notice to **Modosc Designs**.

It will also terminate immediately if you fail to comply with any term of this EULA agreement. Upon such termination, the licenses granted by this EULA agreement will immediately terminate and you agree to stop all access and use of the Software. The provisions that by their nature continue and survive will survive any termination of this EULA agreement.

Governing Law

This EULA agreement, and any dispute arising out of or in connection with this EULA agreement, shall be governed by and construed in accordance with the laws of the state of California.